

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №7
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Обработка временных рядов с помощью нейронных сетей

Цель работы: освоение методов прямого многошагового прогнозирования временных рядов с использованием полносвязных нейронных сетей.

Репозиторий: https://github.com/bickrosss/NN_lab7

Порядок выполнения работы:

Задание 1. Полносвязная сеть для прогнозирования цены закрытия акций Лукойла

В данном задании используется база котировок акций Лукойла за 2016–2019 годы. Реализованы два подхода к прогнозированию: одношаговый и прямой многошаговый на горизонт 10 шагов.

1.1 Загрузка библиотек и подготовка данных

Выполнен импорт необходимых библиотек: NumPy, Pandas, TensorFlow/Keras. Подключены слои Dense, Flatten, Dropout, Conv1D, LSTM, а также нормировщики StandardScaler и MinMaxScaler.

```

# Работа с массивами
import numpy as np

# Работа с таблицами
import pandas as pd

# Классы-конструкторы моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model

# Основные слои
from tensorflow.keras.layers import concatenate, Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, LSTM

# Оптимизаторы
from tensorflow.keras.optimizers import Adam

# Генератор выборки временных рядов
from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator

# Нормировщики
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Загрузка датасетов из облака google
import gdown

# Отрисовка графиков
import matplotlib.pyplot as plt

# Отрисовка графики в ячейке colab
%matplotlib inline

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

# Назначение размера и стиля графиков по умолчанию
from pylab import rcParams
plt.style.use('ggplot')
rcParams['figure.figsize'] = (14, 7)

```

Рисунок 1.1 – Загрузка библиотек

Данные акций Лукойла загружаются двумя файлами (16_17.csv и 18_19.csv) из облачного хранилища и объединяются в один массив. Из общего датасета выделяется канал Close (индекс 3) как целевой ряд для прогнозирования. Итоговый ряд цены закрытия содержит 481 872 наблюдения.

к.	<pre># Загрузка датасетов из облака gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/16_17.csv', None, quiet=True) gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/18_19.csv', None, quiet=True)</pre>
к.	<pre>'18_19.csv'</pre> <pre># Чтение данных в таблицы pandas, столбцы DATE и TIME удаляются за ненадобностью data16_17 = pd.read_csv('16_17.csv', sep=';').drop(columns=['DATE', 'TIME']) data18_19 = pd.read_csv('18_19.csv', sep=';').drop(columns=['DATE', 'TIME']) # Просмотр размерностей print(data16_17.shape) print(data18_19.shape)</pre>
к.	<pre># Объединение двух датасетов в один общий массив data = pd.concat([data16_17, data18_19]).to_numpy() print('Форма общего набора данных:', data.shape)</pre>
	<pre>Форма общего набора данных: (481872, 5)</pre> По условию задачи прогнозируется только цена закрытия (Close), поэтому из общего массива выделяется один соответствующий столбец. <pre># Индексы каналов исходного датасета: Open, Max, Min, Close, Volume CHANNEL_NAMES = ['Open', 'Max', 'Min', 'Close', 'Volume'] CLOSE_INDEX = CHANNEL_NAMES.index('Close') # Отбор только канала Close в виде столбца close_series = data[:, CLOSE_INDEX].reshape(-1, 1) print('Форма ряда цены закрытия:', close_series.shape)</pre> Форма ряда цены закрытия: (481872, 1)

Рисунок 1.2 – Загрузка и объединение датасетов, выделение канала Close

1.2 Визуализация исходного ряда

Построен график фрагмента ряда цены закрытия длиной 300 точек начиная с позиции 100. Ряд демонстрирует случайное блуждание с локальными трендами и отдельными резкими выбросами — характерное поведение внутридневных биржевых котировок.

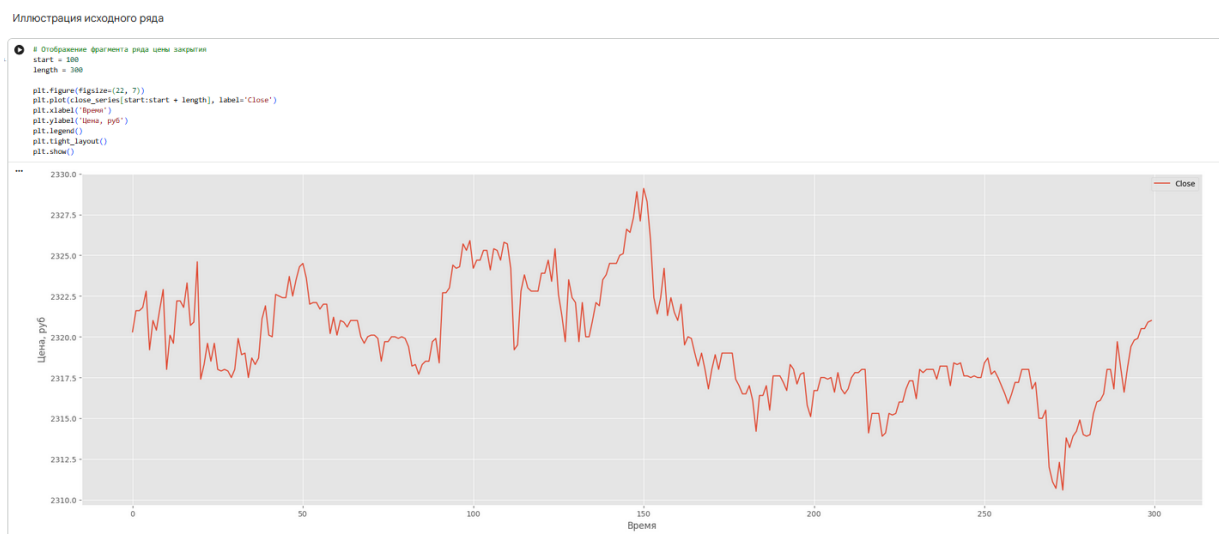


Рисунок 1.3 – Фрагмент ряда цены закрытия акций Лукойла (300 точек)

1.3 Подготовка выборок

Данные разделяются на обучающую и тестовую части в хронологическом порядке. Для устранения пересечения между выборками между ними оставляется зазор длиной в две длины анализируемой последовательности. Размер обучающей части ограничивается последним отрезком ряда. После разделения значения масштабируются с помощью MinMaxScaler, обученного только на тренировочных данных.

Гиперпараметры:

- SEQ_LEN = 60 — длина анализируемой подпоследовательности (1 час истории)
- BATCH_SIZE = 256 — размер батча генератора
- TRAIN_LEN = 100 000 — размер обучающей части
- TEST_LEN = 5 000 — размер тестовой части

<pre># Гиперпараметры подготовки данных SEQ_LEN = 60 BATCH_SIZE = 256 TRAIN_LEN = 100000 TEST_LEN = 5000</pre>	
	<pre># Длина анализируемой подпоследовательности (1 час истории) # Размер батча генератора # Размер обучающей части (последние точки) # Размер тестовой части</pre>
<pre># Разделение по времени с зазором между обучением и тестом. # Берётся последний отрезок ряда: сначала TRAIN_LEN точек на обучение, затем зазор и TEST_LEN на тест. total_len = close_series.shape[0] test_end = total_len test_start = test_end - TEST_LEN train_end = test_start - 2 * SEQ_LEN train_start = train_end - TRAIN_LEN data_train = close_series[train_start:train_end] data_test = close_series[test_start:test_end] # Нормировщик настраивается по обучающим данным scaler = MinMaxScaler() scaler.fit(data_train) # Преобразование обеих частей в float32 – быстрее на GPU и меньше памяти data_train_sc = scaler.transform(data_train).astype(np.float32) data_test_sc = scaler.transform(data_test).astype(np.float32) print('Train:', data_train_sc.shape) print('Test :', data_test_sc.shape)</pre>	
<pre>Train: (100000, 1) Test : (5000, 1)</pre>	

Рисунок 1.4 – Гиперпараметры и подготовка выборок: Train (100000, 1), Test (5000, 1)

Часть 1.1. Полносвязная сеть, прогноз на один шаг вперёд

Стандартный для задач временных рядов подход: на вход сети подаётся окно из SEQ_LEN последних значений, а сеть должна предсказать одно следующее значение. Для формирования батчей используется TimeseriesGenerator.

```

▶ # Генератор обучающей выборки
train_gen = TimeseriesGenerator(data_train_sc, data_train_sc,
                                length=SEQ_LEN,
                                stride=1,
                                sampling_rate=1,
                                batch_size=BATCH_SIZE)

# Генератор тестовой выборки выдаёт всю выборку одним батчем
test_gen = TimeseriesGenerator(data_test_sc, data_test_sc,
                                length=SEQ_LEN,
                                stride=1,
                                sampling_rate=1,
                                batch_size=data_test_sc.shape[0])

x_test, y_test = test_gen[0]
print('x_test:', x_test.shape, 'y_test:', y_test.shape)

... x_test: (4940, 60, 1) y_test: (4940, 1)

```

Рисунок 1.5 – Генераторы выборок для одношагового прогноза: x_test (4940, 60, 1)

1.1.1 Архитектура и обучение

Простая полносвязная сеть с одним скрытым слоем. Входной тензор разворачивается слоем Flatten, затем идёт Dense(150, ReLU) и финальный линейный Dense(1). Всего параметров: 9 301. Модель компилируется с функцией потерь MSE и оптимизатором Adam ($lr = 1e-4$), обучается 8 эпох. Уже к первой эпохе потери резко снижаются и далее остаются стабильно низкими.

```
# Простая полносвязная сеть с одним скрытым слоем
model_dense_1 = Sequential()
model_dense_1.add(Flatten(input_shape=x_test.shape[1:]))
model_dense_1.add(Dense(150, activation='relu'))
model_dense_1.add(Dense(1, activation='linear'))

model_dense_1.compile(loss='mse', optimizer=Adam(learning_rate=1e-4))
model_dense_1.summary()
```

... Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 60)	0
dense (Dense)	(None, 150)	9,150
dense_1 (Dense)	(None, 1)	151

Total params: 9,301 (36.33 KB)
Trainable params: 9,301 (36.33 KB)
Non-trainable params: 0 (0.00 B)

```
# Обучение модели
history_d1 = model_dense_1.fit(train_gen,
                               epochs=8,
                               validation_data=test_gen,
                               verbose=1)
```

Epoch 1/8
391/391 — 3s 5ms/step - loss: 0.0819 - val_loss: 2.7109e-04
Epoch 2/8
391/391 — 1s 3ms/step - loss: 2.1274e-04 - val_loss: 1.4444e-04
Epoch 3/8
391/391 — 1s 3ms/step - loss: 1.0453e-04 - val_loss: 1.1798e-04
Epoch 4/8
391/391 — 2s 5ms/step - loss: 5.7047e-05 - val_loss: 1.0055e-04
Epoch 5/8
391/391 — 1s 3ms/step - loss: 4.5034e-05 - val_loss: 6.2792e-05
Epoch 6/8
391/391 — 1s 3ms/step - loss: 3.4799e-05 - val_loss: 6.4357e-05
Epoch 7/8
391/391 — 1s 3ms/step - loss: 3.4761e-05 - val_loss: 5.7613e-05
Epoch 8/8
391/391 — 1s 3ms/step - loss: 3.2944e-05 - val_loss: 5.5458e-05

Рисунок 1.6 – Архитектура полносвязной сети и лог обучения (8 эпох)

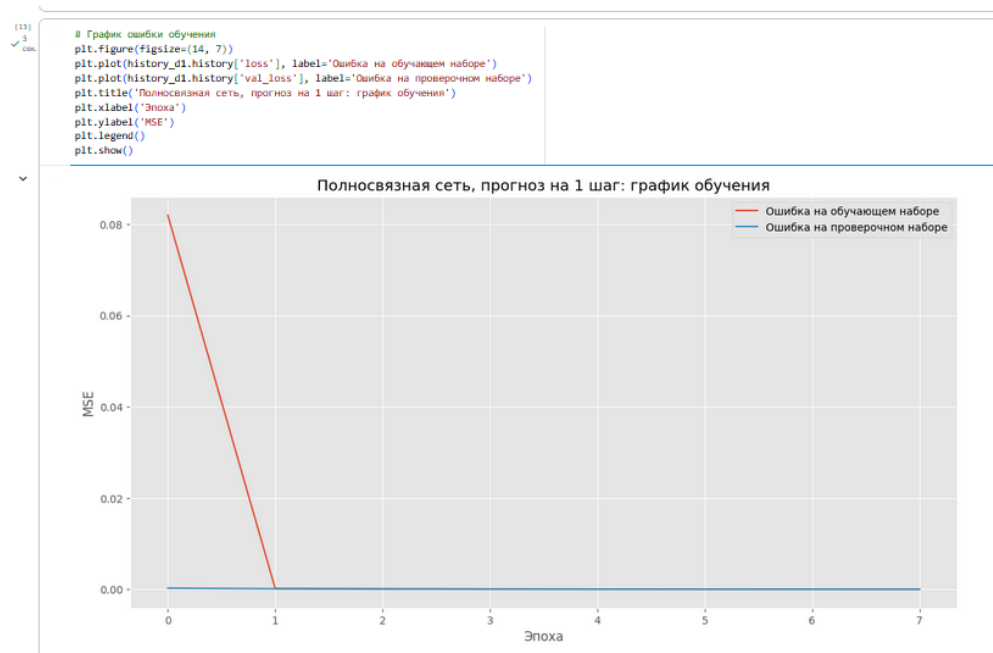


Рисунок 1.7 – График обучения: MSE по эпохам (прогноз на 1 шаг)

1.1.2 Визуализация результата

Прогноз денормализуется и сопоставляется с эталонным рядом на участке длиной 500 точек. Прогнозная кривая (синяя) точно следует за реальными значениями (красная), фиксируя как общий нисходящий тренд в середине периода, так и локальные колебания цены.



Рисунок 1.8 – Сравнение прогноза на 1 шаг с эталонным рядом (500 точек)

Часть 1.2. Полносвязная сеть, прямой прогноз на 10 шагов вперёд

При прямом способе прогноза на горизонт из нескольких шагов сеть выдаёт сразу вектор из 10 значений. Последний слой имеет 10 нейронов, и каждый нейрон отвечает за свою позицию в горизонте прогноза. Для формирования такой выборки требуется собственный генератор окон, поскольку TimeseriesGenerator одиночное число для метки не масштабируется до 10 значений автоматически.

```
.5]
0
сбкл

# Горизонт многошагового прогноза
PREDICT_LAG = 10

# Функция формирования окон признаков и многошаговых меток
def make_multistep(series, seq_len, predict_lag):
    n = series.shape[0] - seq_len - predict_lag + 1
    x = np.stack([series[i:i + seq_len] for i in range(n)]).astype(np.float32)
    y = np.stack([series[i + seq_len:i + seq_len + predict_lag, 0] for i in range(n)]).astype(np.float32)
    return x, y

# Формирование выборок
x_train_m, y_train_m = make_multistep(data_train_sc, SEQ_LEN, PREDICT_LAG)
x_test_m, y_test_m = make_multistep(data_test_sc, SEQ_LEN, PREDICT_LAG)

print('x_train_m:', x_train_m.shape, 'y_train_m:', y_train_m.shape)
print('x_test_m:', x_test_m.shape, 'y_test_m:', y_test_m.shape)

x_train_m: (99931, 60, 1) y_train_m: (99931, 10)
x_test_m: (4931, 60, 1) y_test_m: (4931, 10)
```


Рисунок 1.9 – Формирование многошаговых выборок: x_train_m (99931, 60, 1), y_train_m (99931, 10)

1.2.1 Архитектура и обучение

Та же полносвязная архитектура, но с 10 выходами: Flatten → Dense(150, ReLU) → Dense(10, linear). Всего параметров: 10 660. Модель обучается 8 эпох, потери снижаются аналогично одношаговой модели — резкое падение на первой эпохе, стабилизация далее.

Архитектура и обучение

```

5)
3
200L
# Та же полносвязная архитектура, но с 10 выходами
model_dense_10 = Sequential()
model_dense_10.add(Flatten(input_shape=x_train_m.shape[1:]))
model_dense_10.add(Dense(150, activation='relu'))
model_dense_10.add(Dense(PREDICT_LAG, activation='linear'))

model_dense_10.compile(loss='mse', optimizer=Adam(learning_rate=1e-4))
model_dense_10.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 60)	0
dense_2 (Dense)	(None, 150)	9,150
dense_3 (Dense)	(None, 10)	1,510

Total params: 10,660 (41.64 KB)
Trainable params: 10,660 (41.64 KB)
Non-trainable params: 0 (0.00 B)

```

7)
7
200L
# Обучение модели
history_d10 = model_dense_10.fit(x_train_m, y_train_m,
                                validation_data=(x_test_m, y_test_m),
                                epochs=8,
                                batch_size=BATCH_SIZE,
                                verbose=1)

```

```

Epoch 1/8
391/391 ————— 3s 5ms/step - loss: 0.0371 - val_loss: 2.0594e-04
Epoch 2/8
391/391 ————— 1s 3ms/step - loss: 8.4347e-05 - val_loss: 1.6645e-04
Epoch 3/8
391/391 ————— 1s 4ms/step - loss: 6.8137e-05 - val_loss: 1.4352e-04
Epoch 4/8
391/391 ————— 1s 3ms/step - loss: 5.9717e-05 - val_loss: 1.2658e-04
Epoch 5/8
391/391 ————— 1s 2ms/step - loss: 5.1601e-05 - val_loss: 1.0354e-04
Epoch 6/8
391/391 ————— 1s 3ms/step - loss: 4.6347e-05 - val_loss: 9.2687e-05
Epoch 7/8
391/391 ————— 1s 3ms/step - loss: 4.4038e-05 - val_loss: 8.8403e-05
Epoch 8/8
391/391 ————— 1s 2ms/step - loss: 4.2383e-05 - val_loss: 8.3225e-05

```

Рисунок 1.10 – Архитектура многошаговой модели и лог обучения (8 эпох)

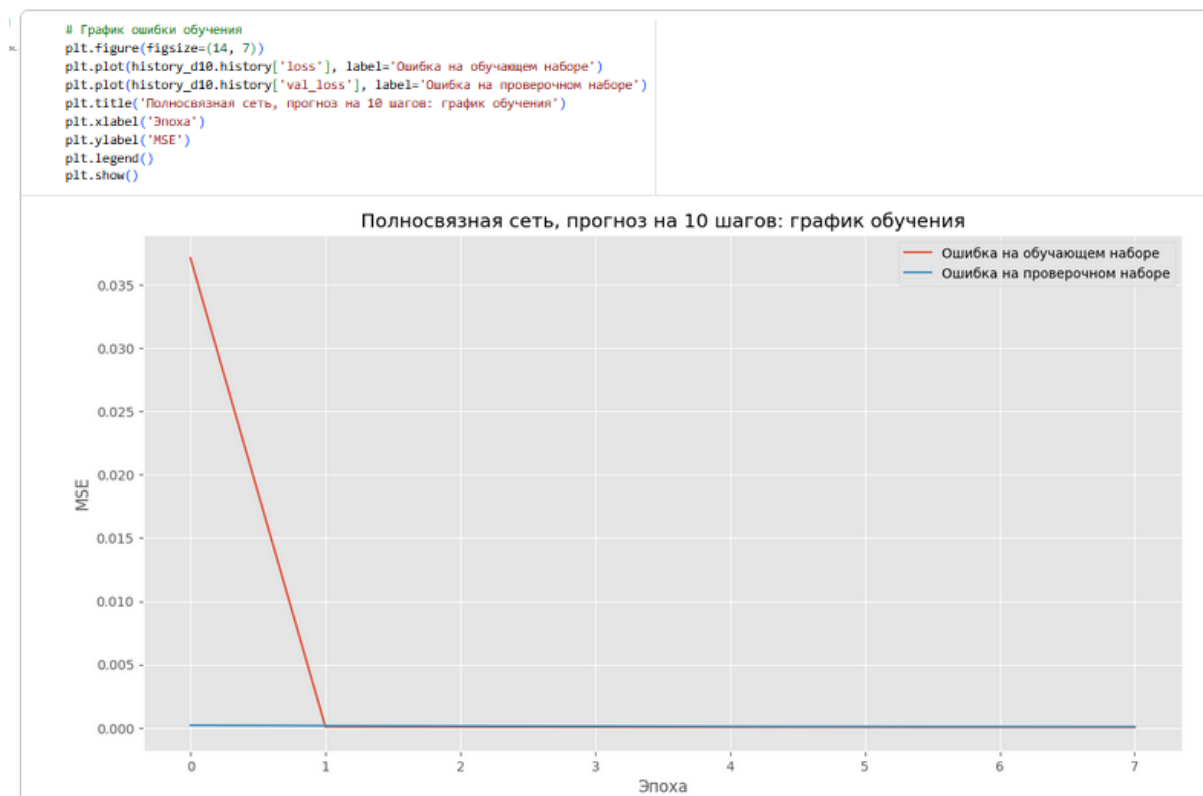


Рисунок 1.11 – График обучения: MSE по эпохам (прогноз на 10 шагов)

1.2.2 Графики сравнения по 10 шагам

Каждый из 10 графиков соответствует одной позиции в горизонте прогноза: первый график — прогноз на 1 шаг вперёд, второй — на 2 шага вперёд, и так до 10. Эталонный ряд для каждого графика сдвигается во времени на соответствующее число шагов, чтобы сравнение проводилось корректно. Денормализованный прогноз и эталон содержат по 4 931 точке.

```

# Получение прогноза и его обратная нормализация
y_pred_m_sc = model_dense_10.predict(x_test_m)

# Канал был один, поэтому inverse_transform применяется к каждому шагу прогноза отдельно
y_pred_m = scaler.inverse_transform(y_pred_m_sc.reshape(-1, 1)).reshape(y_pred_m_sc.shape)
y_true_m = scaler.inverse_transform(y_test_m.reshape(-1, 1)).reshape(y_test_m.shape)

print('Денормализованный прогноз:', y_pred_m.shape)
print('Денормализованный эталон:', y_true_m.shape)

```

155/155 ————— 1s 3ms/step
Денормализованный прогноз: (4931, 10)
Денормализованный эталон : (4931, 10)

Рисунок 1.12 – Денормализация прогноза: размерности (4931, 10)

```

18)
36
сек.

# 10 отдельных графиков (10 рядов на разных осях)
start = 0
length = 500

fig, axes = plt.subplots(PREDICT_LAG, 1, figsize=(22, 4 * PREDICT_LAG))

for step in range(PREDICT_LAG):
    ax = axes[step]
    ax.plot(y_true_m[start:start + length, step], label='Эталон')
    ax.plot(y_pred_m[start:start + length, step], label='Прогноз')
    ax.set_title(f'Полносвязная сеть: шаг прогноза +{step + 1}')
    ax.set_xlabel('Время')
    ax.set_ylabel('Цена, руб')
    ax.legend()

plt.tight_layout()
plt.show()

```

Рисунок 1.13 – Код построения 10 графиков сравнения



Рисунок 1.14 – Шаги прогноза +1 и +2



Рисунок 1.15 – Шаги прогноза +3 и +4

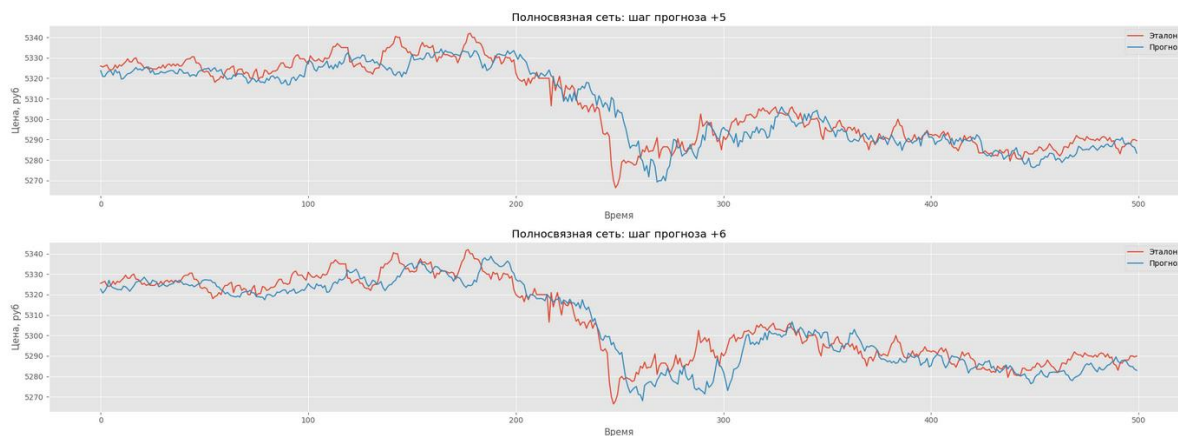


Рисунок 1.16 – Шаги прогноза +5 и +6

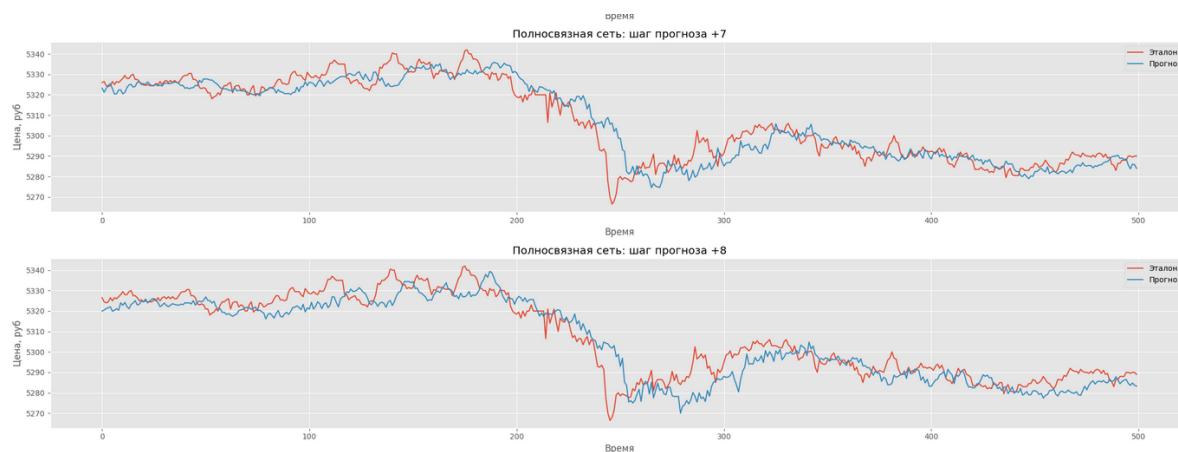


Рисунок 1.17 – Шаги прогноза +7 и +8

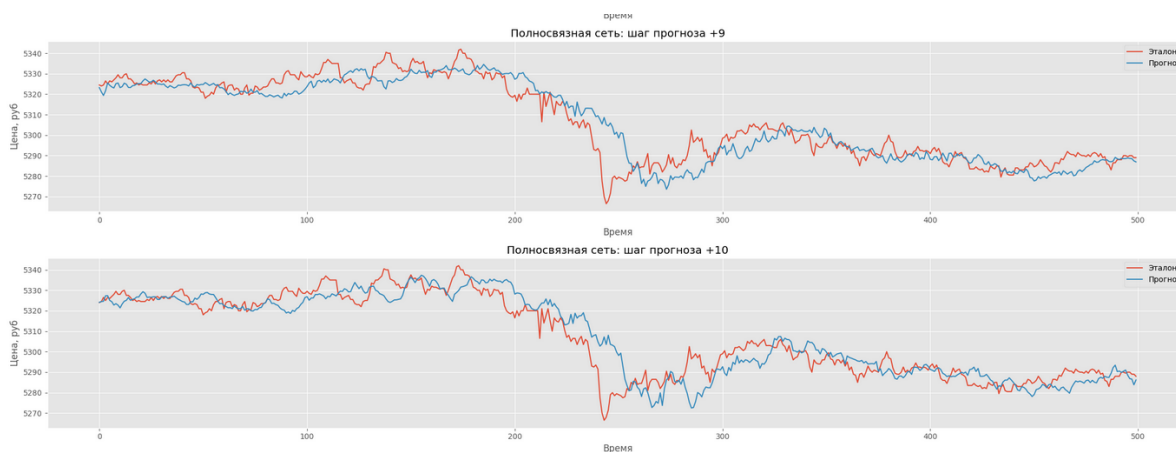


Рисунок 1.18 – Шаги прогноза +9 и +10

1.2.3 Выводы по заданию 1

В ходе выполнения задания реализованы два подхода к прогнозированию цены закрытия акций Лукойла на основе полносвязных нейронных сетей.

Одношаговая модель демонстрирует высокую точность: прогнозная кривая практически совпадает с эталоном, фиксируя локальные колебания цены. Это объясняется тем, что соседние значения биржевого ряда сильно коррелированы.

Многошаговая модель обеспечивает прогноз сразу на 10 шагов вперёд в рамках одного прохода. Анализ графиков по шагам показывает, что качество прогноза постепенно снижается с увеличением горизонта: прогнозная кривая на шагах +1, +2 вплотную следует за эталоном, тогда как на шагах +9, +10 расхождение несколько возрастает, особенно в зонах резких ценовых движений. Тем не менее общий тренд и диапазон значений сохраняются на всех 10 шагах.

Задание 1.2. Реализация на PyTorch: одношаговый и многошаговый прогноз

Повторная реализация задания с использованием фреймворка PyTorch. Вместо Keras-генераторов применяется собственный ленивый Dataset класс SeriesWindowDataset. Обучение выполняется на GPU (CUDA), что обеспечивает значительное ускорение на больших объёмах данных.

1.2.1 Загрузка библиотек

Импортированы PyTorch, torch.nn, DataLoader и TensorDataset. Выбор устройства выполняется автоматически: при наличии GPU используется CUDA, иначе CPU. Зафиксированы случайные зёрна для воспроизводимости результатов (SEED = 42).

```
[ ]
# Работа с массивами
import numpy as np

# Работа с таблицами
import pandas as pd

# PyTorch
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

# Нормировщики
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Загрузка датасетов из облака google
import gdown

# Отрисовка графиков
import matplotlib.pyplot as plt

# Отрисовка графики в ячейке colab
%matplotlib inline

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

# Выбор устройства для обучения
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Устройство:', device)

Устройство: cuda

[ ]
# Назначение размера и стиля графиков по умолчанию
from pylab import rcParams
plt.style.use('ggplot')
rcParams['figure.figsize'] = (14, 7)

# Фиксация случайных значений для воспроизводимости
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

Рисунок 1.19 – Загрузка библиотек PyTorch и фиксация случайных значений

1.2.2 Загрузка данных

Используется та же база котировок акций Лукойла за 2016–2019 годы. Данные загружаются двумя файлами и объединяются в массив формы (481872, 5). Из общего массива выделяется канал Close (индекс 3) в виде столбца формы (481872, 1).

```

1 # Загрузка датасетов из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/16_17.csv', None, quiet=True)
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/18_19.csv', None, quiet=True)

'18_19.csv'

1 # Чтение данных в таблицы pandas, столбцы DATE и TIME удаляются за ненадобностью
data16_17 = pd.read_csv('16_17.csv', sep=';').drop(columns=['DATE', 'TIME'])
data18_19 = pd.read_csv('18_19.csv', sep=';').drop(columns=['DATE', 'TIME'])

# Просмотр размерностей
print(data16_17.shape)
print(data18_19.shape)

(263925, 5)
(217947, 5)

1 # Объединение двух датасетов в один общий массив
data = pd.concat([data16_17, data18_19]).to_numpy()
print('Форма общего набора данных:', data.shape)

Форма общего набора данных: (481872, 5)

По условию задачи прогнозируется только цена закрытия (Close), поэтому из общего массива выделяется один соответствующий столбец.

1 # Индексы каналов исходного датасета: Open, Max, Min, Close, Volume
CHANNEL_NAMES = ['Open', 'Max', 'Min', 'Close', 'Volume']
CLOSE_INDEX = CHANNEL_NAMES.index('Close')

# Отбор только канала Close в виде столбца
close_series = data[:, CLOSE_INDEX].reshape(-1, 1)
print('Форма ряда цены закрытия:', close_series.shape)

Форма ряда цены закрытия: (481872, 1)

```

Рисунок 1.20 – Загрузка и объединение датасетов, выделение канала Close

1.2.3 Визуализация исходного ряда

Построен график фрагмента ряда цены закрытия длиной 300 точек начиная с позиции 100. Характер ряда совпадает с предыдущей реализацией: случайное блуждание с локальными трендами.

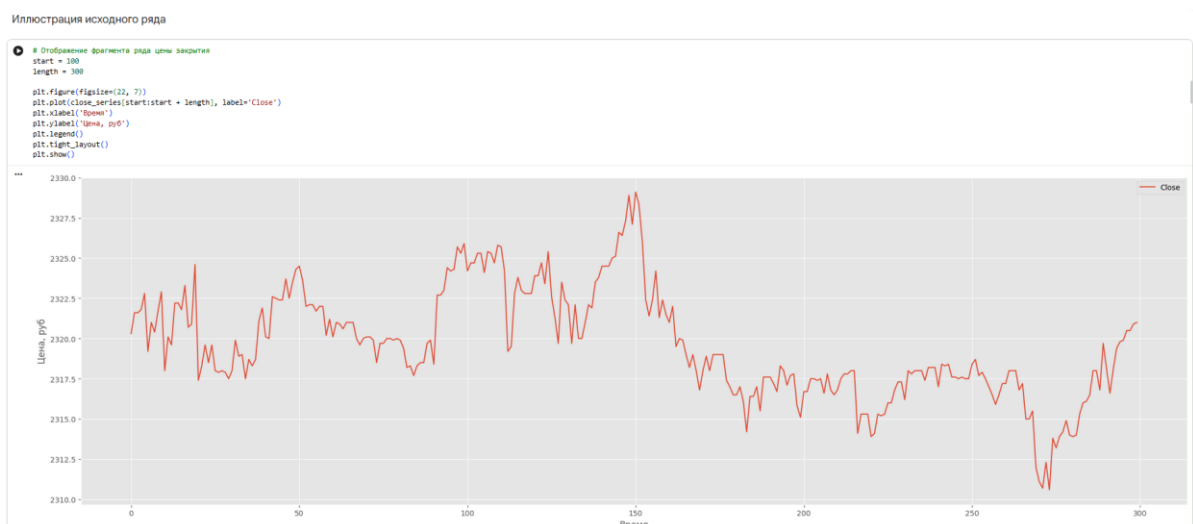


Рисунок 1.21 – Фрагмент ряда цены закрытия (300 точек)

1.2.4 Подготовка выборки

Гиперпараметры скорректированы относительно Keras-реализации:

- SEQ_LEN = 60 — длина анализируемой подпоследовательности

- BATCH_SIZE = 64 — размер батча
- TEST_LEN = 30 000 — размер тестовой части
- TRAIN_LEN = close_series.shape[0] – TEST_LEN — размер обучающей части

Нормировщик MinMaxScaler обучается только на тренировочных данных. Обе части приводятся к типу float32. Результат: Train (451872, 1), Test (29880, 1).

```
# Гиперпараметры подготовки данных
SEQ_LEN = 60 # Длина анализируемой подпоследовательности
BATCH_SIZE = 64 # Размер батча
TEST_LEN = 30000 # Размер тестовой части
TRAIN_LEN = close_series.shape[0] - TEST_LEN # Размер обучающей части

# Разделение по времени с зазором между обучением и тестом
data_train = close_series[:TRAIN_LEN]
data_test = close_series[TRAIN_LEN + 2 * SEQ_LEN:]

# Нормировщик настраивается по обучающим данным
scaler = MinMaxScaler()
scaler.fit(data_train)

# Преобразование обеих частей
data_train_sc = scaler.transform(data_train).astype(np.float32)
data_test_sc = scaler.transform(data_test).astype(np.float32)

print('Train:', data_train_sc.shape)
print('Test :', data_test_sc.shape)

Train: (451872, 1)
Test : (29880, 1)
```

Рисунок 1.22 – Гиперпараметры и подготовка выборок: Train (451872,1), Test (29880,1)

1.2.5 Ленивый Dataset и DataLoader

Для эффективной работы с большими рядами реализован класс SeriesWindowDataset, наследующий от torch.utils.data.Dataset. Он хранит один общий тензор ряда и нарезает окна непосредственно при обращении по индексу (`__getitem__`), не материализуя весь массив в памяти. Поверх датасетов создаются DataLoader с перемешиванием для обучающего набора и без — для тестового.


```

# Ленивый набор данных, который нарезает окна непосредственно при обращении
class SeriesWindowDataset(torch.utils.data.Dataset):
    def __init__(self, series, seq_len, predict_lag):
        # Хранится один общий тензор ряда, окна вырезаются по индексу
        self.series = torch.from_numpy(series)
        self.seq_len = seq_len
        self.predict_lag = predict_lag
        self.length = series.shape[0] - seq_len - predict_lag + 1
        if self.length <= 0:
            raise ValueError('Слишком короткий ряд для заданных параметров')

    def __len__(self):
        return self.length

    def __getitem__(self, idx):
        x = self.series[idx:idx + self.seq_len]
        if self.predict_lag == 1:
            y = self.series[idx + self.seq_len]
        else:
            # Один канал, поэтому берётся столбец 0
            y = self.series[idx + self.seq_len:idx + self.seq_len + self.predict_lag, 0]
        return x, y

# Горизонт многошагового прогноза
PREDICT_LAG = 10

# Наборы данных для одношагового и многошагового прогноза
train_ds_1 = SeriesWindowDataset(data_train_sc, SEQ_LEN, predict_lag=1)
test_ds_1 = SeriesWindowDataset(data_test_sc, SEQ_LEN, predict_lag=1)
train_ds_10 = SeriesWindowDataset(data_train_sc, SEQ_LEN, predict_lag=PREDICT_LAG)
test_ds_10 = SeriesWindowDataset(data_test_sc, SEQ_LEN, predict_lag=PREDICT_LAG)

print('1 шаг train:', len(train_ds_1), 'test:', len(test_ds_1))
print('10 шагов train:', len(train_ds_10), 'test:', len(test_ds_10))

... 1 шаг train: 451812 test: 29820
    10 шагов train: 451803 test: 29811

```

Рисунок 1.23 – Класс SeriesWindowDataset и создание DataLoader'ов

```

# DataLoader'ы поверх ленивых наборов
def make_loaders(train_ds, test_ds, batch_size=BATCH_SIZE):
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader

train_loader_1, test_loader_1 = make_loaders(train_ds_1, test_ds_1)
train_loader_10, test_loader_10 = make_loaders(train_ds_10, test_ds_10)

```

Рисунок 1.24 – Создание загрузчиков данных для одно- и многошагового прогноза

1.2.6 Сервисные функции обучения и оценки

Реализованы универсальные функции, используемые для всех моделей: train_one_epoch (одна эпоха обучения с накоплением градиентов), evaluate (оценка ошибки на тестовой выборке без градиентов), fit (полный цикл обучения с накоплением истории), predict_all (прогноз по всему набору данных батчами) и plot_history (визуализация кривых обучения).

```

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0.0
    total_n = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0)
        total_n += xb.size(0)
    return total_loss / total_n

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss = 0.0
    total_n = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        pred = model(xb)
        loss = criterion(pred, yb)
        total_loss += loss.item() * xb.size(0)
        total_n += xb.size(0)
    return total_loss / total_n

def fit(model, train_loader, test_loader, epochs, lr=1e-4):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.MSELoss()
    history = {'loss': [], 'val_loss': []}
    for epoch in range(1, epochs + 1):
        train_loss = train_one_epoch(model, train_loader, optimizer, criterion)
        val_loss = evaluate(model, test_loader, criterion)
        history['loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        print(f'Эпоха {epoch:3d}/{epochs}   loss = {train_loss:.6f}   val_loss = {val_loss:.6f}')
    return history

```

Рисунок 1.25 – Сервисные функции: train_one_epoch, evaluate, fit

```

@torch.no_grad()
def predict_all(model, dataset, batch_size=256):
    # Прогноз по всему набору данных батчами, возвращаются numpy-массивы (y_pred, y_true)
    model.eval()
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=False)
    preds, trues = [], []
    for xb, yb in loader:
        xb = xb.to(device)
        preds.append(model(xb).cpu().numpy())
        trues.append(yb.numpy())
    return np.concatenate(preds, axis=0), np.concatenate(trues, axis=0)

def plot_history(history, title):
    plt.figure(figsize=(14, 7))
    plt.plot(history['loss'], label='Ошибка на обучающем наборе')
    plt.plot(history['val_loss'], label='Ошибка на проверочном наборе')
    plt.title(title)
    plt.xlabel('Эпоха')
    plt.ylabel('MSE')
    plt.legend()
    plt.show()

```

Рисунок 1.26 – Функции predict_all и plot_history

Часть 1.2а. PyTorch: полносвязная сеть, прогноз на 1 шаг

Простейшая архитектура: разворачивание окна в одномерный вектор, скрытый полносвязный слой и линейный выход с одним нейроном. Модель реализована как класс DenseModel1, наследующий от nn.Module.

```
# Простая полносвязная сеть с одним выходом
class DenseModel1(nn.Module):
    def __init__(self, seq_len):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(seq_len, 150),
            nn.ReLU(),
            nn.Linear(150, 1),
        )

    def forward(self, x):
        return self.net(x)

model_dense_1 = DenseModel1(SEQ_LEN).to(device)
print(model_dense_1)

*** DenseModel1(
  (net): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=60, out_features=150, bias=True)
    (2): ReLU()
    (3): Linear(in_features=150, out_features=1, bias=True)
  )
)
```

Рисунок 1.27 – Архитектура DenseModel1: Flatten → Linear(60→150) → ReLU → Linear(150→1)

Обучение

Модель обучается 20 эпох с оптимизатором Adam ($lr = 1e-4$) и функцией потерь MSELoss. Уже к первой эпохе потери падают с 0.000130 до ~ 0.000003 и далее стабилизируются на уровне порядка $1e-6$ – $1e-5$.

# Обучение			
history_d1 = fit(model_dense_1, train_loader_1, test_loader_1, epochs=20, lr=1e-4)			
Эпоха	1/20	loss = 0.000130	val_loss = 0.000008
Эпоха	2/20	loss = 0.000003	val_loss = 0.000010
Эпоха	3/20	loss = 0.000003	val_loss = 0.000003
Эпоха	4/20	loss = 0.000002	val_loss = 0.000002
Эпоха	5/20	loss = 0.000002	val_loss = 0.000004
Эпоха	6/20	loss = 0.000002	val_loss = 0.000002
Эпоха	7/20	loss = 0.000002	val_loss = 0.000006
Эпоха	8/20	loss = 0.000002	val_loss = 0.000005
Эпоха	9/20	loss = 0.000002	val_loss = 0.000002
Эпоха	10/20	loss = 0.000001	val_loss = 0.000001
Эпоха	11/20	loss = 0.000001	val_loss = 0.000002
Эпоха	12/20	loss = 0.000001	val_loss = 0.000003
Эпоха	13/20	loss = 0.000001	val_loss = 0.000002
Эпоха	14/20	loss = 0.000001	val_loss = 0.000001
Эпоха	15/20	loss = 0.000001	val_loss = 0.000001
Эпоха	16/20	loss = 0.000001	val_loss = 0.000001
Эпоха	17/20	loss = 0.000001	val_loss = 0.000001
Эпоха	18/20	loss = 0.000001	val_loss = 0.000009
Эпоха	19/20	loss = 0.000001	val_loss = 0.000002
Эпоха	20/20	loss = 0.000001	val_loss = 0.000002

Рисунок 1.28 – Лог обучения DenseModel1 (20 эпох)

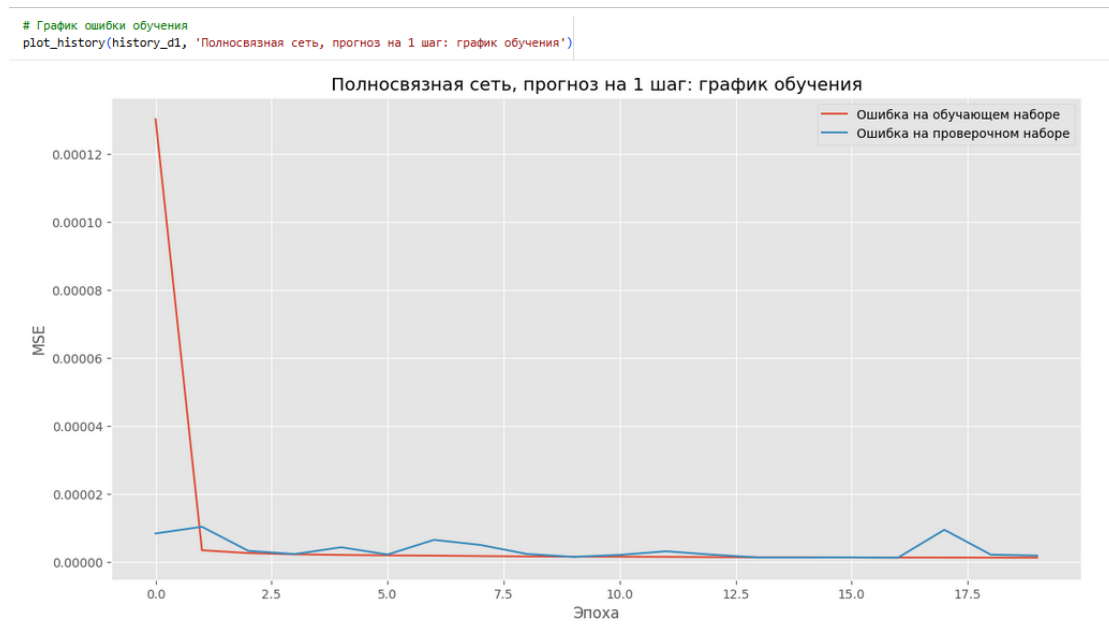


Рисунок 1.29 – График обучения: MSE по эпохам

Визуализация результата

Прогноз денормализуется и сопоставляется с эталонным рядом на участке 500 точек. Прогнозная кривая (синяя) плотно следует за реальными значениями (красная), фиксируя как нисходящий тренд в середине периода, так и локальные колебания. Реализация на PyTorch даёт результаты, сопоставимые с Keras-версией.

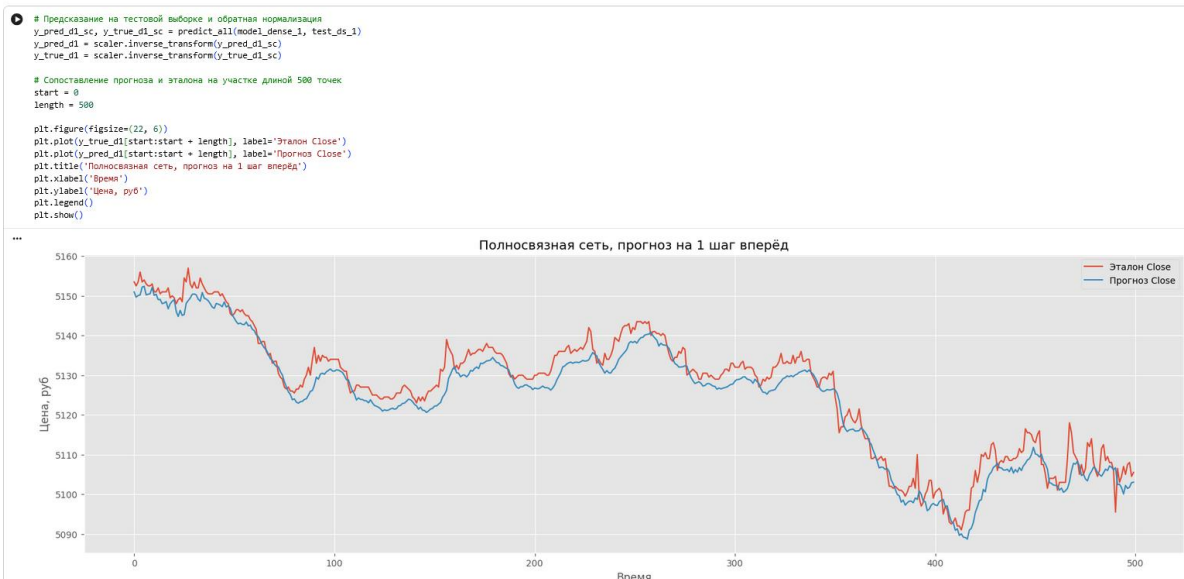


Рисунок 1.30 – Сравнение прогноза на 1 шаг с эталонным рядом (500 точек, PyTorch)

Часть 1.26. PyTorch: полносвязная сеть, прямой прогноз на 10 шагов

При прямом способе прогноза на горизонт из 10 шагов сеть выдаёт сразу вектор из 10 значений. Класс DenseModel10 идентичен DenseModel1, но последний линейный слой имеет 10 выходов.

```

# Та же архитектура, но с 10 выходами
class DenseModel10(nn.Module):
    def __init__(self, seq_len, predict_lag):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(seq_len, 150),
            nn.ReLU(),
            nn.Linear(150, predict_lag),
        )

    def forward(self, x):
        return self.net(x)

model_dense_10 = DenseModel10(SEQ_LEN, PREDICT_LAG).to(device)
print(model_dense_10)

```

```

DenseModel10(
  (net): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=60, out_features=150, bias=True)
    (2): ReLU()
    (3): Linear(in_features=150, out_features=10, bias=True)
  )
)

```

Рисунок 1.31 – Архитектура DenseModel10: Linear(150→10), вывод структуры модели

Обучение

Модель обучается 20 эпох. Потери с первой эпохи снижаются с 0.001574 до уровня $\sim 3\text{e-}4$ и далее стабилизируются. Ошибка на тестовой выборке (val_loss) удерживается в диапазоне $4\text{e-}6 - 1.4\text{e-}5$.

```
# Обучение
history_d10 = fit(model_dense_10, train_loader_10, test_loader_10, epochs=20, lr=1e-4)
```

Эпоха	1/20	loss = 0.001574	val_loss = 0.000009
Эпоха	2/20	loss = 0.000005	val_loss = 0.000006
Эпоха	3/20	loss = 0.000004	val_loss = 0.000006
Эпоха	4/20	loss = 0.000004	val_loss = 0.000005
Эпоха	5/20	loss = 0.000004	val_loss = 0.000005
Эпоха	6/20	loss = 0.000003	val_loss = 0.000005
Эпоха	7/20	loss = 0.000003	val_loss = 0.000004
Эпоха	8/20	loss = 0.000003	val_loss = 0.000004
Эпоха	9/20	loss = 0.000003	val_loss = 0.000014
Эпоха	10/20	loss = 0.000003	val_loss = 0.000004
Эпоха	11/20	loss = 0.000003	val_loss = 0.000004
Эпоха	12/20	loss = 0.000003	val_loss = 0.000005
Эпоха	13/20	loss = 0.000003	val_loss = 0.000006
Эпоха	14/20	loss = 0.000003	val_loss = 0.000004
Эпоха	15/20	loss = 0.000003	val_loss = 0.000008
Эпоха	16/20	loss = 0.000003	val_loss = 0.000006
Эпоха	17/20	loss = 0.000003	val_loss = 0.000005
Эпоха	18/20	loss = 0.000003	val_loss = 0.000005
Эпоха	19/20	loss = 0.000003	val_loss = 0.000004
Эпоха	20/20	loss = 0.000003	val_loss = 0.000005

Рисунок 1.32 – Лог обучения DenseModel10 (20 эпох)

```
# График ошибки обучения
plot_history(history_d10, 'Полносвязная сеть, прогноз на 10 шагов: график обучения')
```

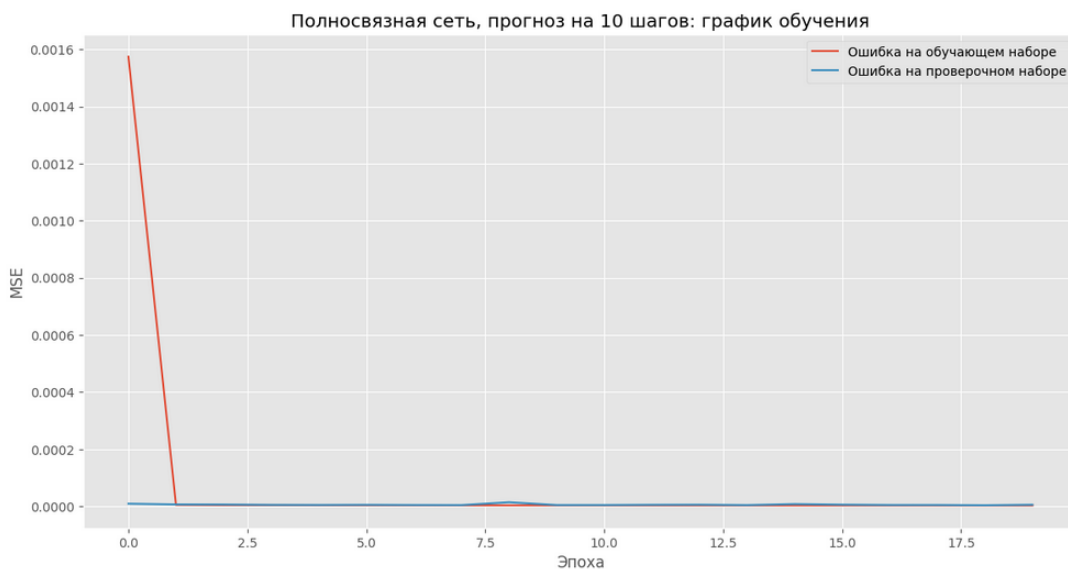


Рисунок 1.33 – График обучения: MSE по эпохам (прогноз на 10 шагов, PyTorch)

Графики сравнения по 10 шагам

Каждый из 10 графиков соответствует одной позиции в горизонте прогноза. Эталонный ряд для каждого графика — фактические значения именно той позиции, на которую сеть делает прогноз. Денормализованный прогноз и эталон содержат по 29 811 точек $\times$ 10 шагов.

```
# Получение прогноза и его обратная нормализация
y_pred_m_sc, y_true_m_sc = predict_all(model_dense_10, test_ds_10)

# Канал был один, поэтому inverse_transform применяется к развёрнутому массиву и затем возвращается форма
y_pred_m = scaler.inverse_transform(y_pred_m_sc.reshape(-1, 1)).reshape(y_pred_m_sc.shape)
y_true_m = scaler.inverse_transform(y_true_m_sc.reshape(-1, 1)).reshape(y_true_m_sc.shape)

print('Денормализованный прогноз:', y_pred_m.shape)
print('Денормализованный эталон :', y_true_m.shape)
```

Денормализованный прогноз: (29811, 10)
Денормализованный эталон : (29811, 10)

Рисунок 1.34 – Денормализация многошагового прогноза: размерности (29811, 10)

1.2.7 Итоговые выводы по заданию 1.2

Реализация на PyTorch подтвердила результаты Keras-версии. Ленивый Dataset позволяет эффективно работать с большими рядами без материализации всего массива окон в памяти. Использование GPU (CUDA) обеспечивает значительное ускорение обучения.

Одношаговая модель достигает MSE порядка $1e-6$ на тестовой выборке и дает высокое визуальное соответствие эталону. Многошаговая модель (10 шагов) сохраняет точность на ближних горизонтах прогноза, при этом расхождение с эталоном незначительно возрастает на дальних шагах. Оба подхода подтверждают применимость полносвязных нейронных сетей для краткосрочного прогнозирования внутридневных биржевых котировок.

Задание 2. Предсказание временного ряда с расширенными признаками и многомасштабным окном

В данном задании используется база акций Лукойла. Для улучшения качества предсказания временного ряда к исходным пяти каналам (OPEN, MAX, MIN, CLOSE, VOLUME) добавляются расширенные признаки, а входной сигнал формируется с использованием многомасштабного окна просмотра в прошлое.

2.1 Подготовка среды и загрузка данных

Выполнен импорт необходимых библиотек: NumPy, Pandas, TensorFlow/Keras, sklearn. Подключены слои Conv1D, LSTM, Dense, Dropout, BatchNormalization, Flatten, а также нормировщики StandardScaler и MinMaxScaler. Загружены два датасета акций Лукойла за периоды 2016–2017 и 2018–2019 годов.

```
[ ] # Работа с массивами
import numpy as np

# Работа с таблицами
import pandas as pd

# Классы-конструкторы моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model

# Основные слои
from tensorflow.keras.layers import concatenate, Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, LSTM, MaxPooling1D

# Оптимизаторы
from tensorflow.keras.optimizers import Adam

# Генератор выборки временных рядов
from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator

# Нормировщики
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Колбэк
from tensorflow.keras.callbacks import LearningRateScheduler

# Загрузка датасетов из облака google
import gdown

# Отрисовка графиков
import matplotlib.pyplot as plt

# Отрисовка графики в ячейке colab
%matplotlib inline

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

[ ] # Назначение размера и стиля графиков по умолчанию
from pylab import rcParams
plt.style.use('ggplot')
rcParams['figure.figsize'] = (14, 7)
```

Рисунок 2.1 – Подключение библиотек и настройка среды

```
# Загрузка датасетов из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/16_17.csv', None, quiet=True)
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/18_19.csv', None, quiet=True)

'18_19.csv'

# Чтение данных в таблицы с удалением ненужных столбцов
data16_17 = pd.read_csv('16_17.csv', sep=';').drop(columns=['DATE', 'TIME'])
data18_19 = pd.read_csv('18_19.csv', sep=';').drop(columns=['DATE', 'TIME'])

# Просмотр размерностей получившихся таблиц
print(data16_17.shape)
print(data18_19.shape)

(263925, 5)
(217947, 5)

# Создание общего набора данных из двух датасетов
data = pd.concat([data16_17, data18_19]) # Объединение датасетов
data = data.reset_index(drop=True)      # Обнуление индексов

# Проверка формы данных
print(data.shape)

(481872, 5)
```


Рисунок 2.2 – Загрузка и объединение датасетов акций Лукойла

После объединения двух датасетов итоговая таблица содержит 481 872 строки и 5 столбцов (OPEN, MAX, MIN, CLOSE, VOLUME).

2.2 Расширение исходного сигнала

К пяти исходным каналам добавляется набор производных характеристик. Все новые столбцы вычисляются непосредственно в DataFrame и сохраняют выравнивание по строкам с исходными данными.

Попарные разности, модули разностей и произведения каналов. Перебираются все упорядоченные пары столбцов исходных каналов, для каждой пары формируются три новых столбца:

- разность: $\text{col}[i] - \text{col}[j]$
- модуль разности: $|\text{col}[i] - \text{col}[j]|$
- произведение: $\text{col}[i] \times \text{col}[j]$

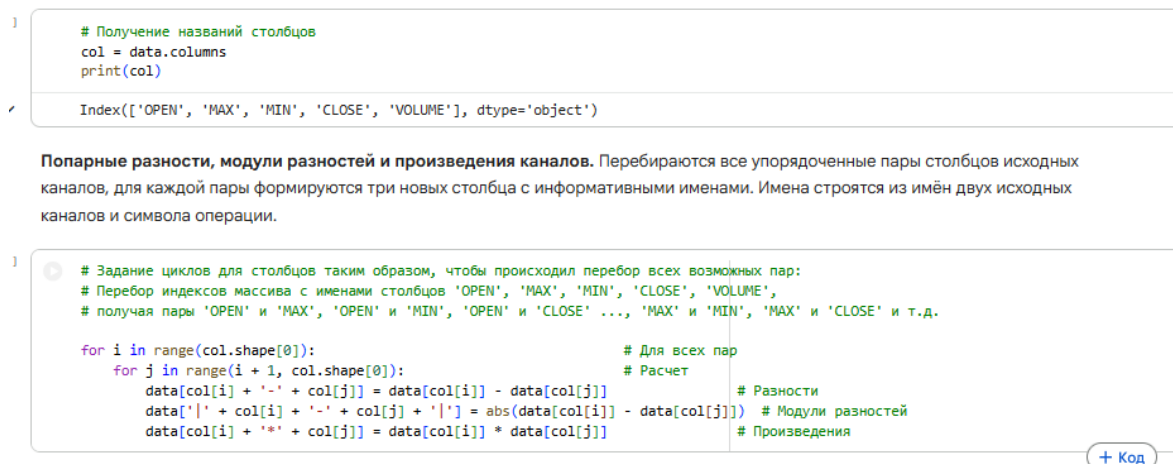


Рисунок 2.3 – Расширение исходного сигнала: попарные разности, модули и произведения каналов

Для каждого исходного канала вычисляются ещё три величины: обратное значение по формуле $1/(x + 1e-3)$ (константа в знаменателе исключает деление на ноль), первая производная как разность соседних значений и вторая производная по схеме $x[n] - 2 \cdot x[n-1] + x[n-2]$. Производные определены не на всех индексах, поэтому первые одна-две позиции соответствующих столбцов остаются NaN.

6

Для каждого столбца "OPEN", "MAX", "MIN", "CLOSE", "VOLUME" расчет:
for i in cols:
Обратные значения. In-3 в формуле нулю, чтобы случайно не разделить на 0
data["Обратный" + i] = 1 / (data[i] + In-3)
Создание пустого столбца
data["Производная от " + i] = np.nan
При помощи срезов расчет первых производных, .reset_index(drop=True) нужен для корректных расчетов
data["Производная от " + i][1:] = data[i][1:].reset_index(drop=True) - data["Обратный" + i][1:].reset_index(drop=True)
Создание пустого столбца
data["Вторые производные от " + i] = np.nan
При помощи срезов расчет вторых производных
data["Вторые производные от " + i][2:] = data[i][2:].reset_index(drop=True) - 2 * data[i][1:].reset_index(drop=True) + data[i][:-2].reset_index(drop=True)

7

Просмотр результатов
data

	OPEN	MAX	MIN	CLOSE	VOLUME	OPEN- MAX	[OPEN- MAX]	OPEN*MAX	OPEN- MIN	[OPEN- MIN]	...	Вторые производные от or MAX	Обратный RIN	Производная от RIN	Вторые производные от RIN	Обратный CLOSE	Производная от CLOSE	Вторые производные от CLOSE	Обратный VOLUME	Производная от VOLUME	Вторые производные от VOLUME	
0	2351.0	2355.8	2350.0	2350.0	2547	-4.8	4.8	553845.80	1.0	1.0	---	NaN	0.000425	NaN	NaN	NaN	0.000425	NaN	NaN	0.000393	NaN	NaN
1	2352.9	2355.7	2350.0	2355.7	195	-2.8	2.8	5542726.53	2.9	2.9	---	NaN	0.000425	NaN	NaN	NaN	0.000425	NaN	NaN	0.005128	-2352.0	NaN
2	2355.6	2356.0	2351.4	2354.1	257	-0.4	0.4	5548793.90	4.2	4.2	---	0.4	0.000425	1.4	1.4	0.000425	-1.6	-7.3	0.003891	62.0	2614.0	
3	2354.5	2355.0	2351.2	2353.7	763	-0.5	0.5	5546847.50	3.3	3.3	---	-1.3	0.000425	-0.2	-1.6	0.000425	-0.4	1.2	0.001311	506.0	444.0	
4	2353.1	2353.9	2353.1	2353.6	231	-0.8	0.8	5538962.09	0.0	0.0	---	-0.1	0.000425	1.9	2.1	0.000425	-0.1	0.3	0.004329	-532.0	-1038.0	
...	...	...	...	...	...	...	...	...	...	...	---	...	...	...	...	...	...	...	...	...	...	
401867	5183.5	5183.5	5183.5	5183.5	31523	0.0	0.0	26889872.25	0.0	0.0	---	-11.5	0.000193	16.0	16.0	0.000193	11.5	10.0	0.000032	29174.0	29480.0	
401868	5183.5	5183.5	5183.5	5183.5	5090	0.0	0.0	26889872.25	0.0	0.0	---	-11.5	0.000193	0.0	-18.0	0.000193	0.0	-11.5	0.000196	-26433.0	-55607.0	
401869	5183.5	5183.5	5183.5	5183.5	230	0.0	0.0	26889872.25	0.0	0.0	---	0.0	0.000193	0.0	0.0	0.000193	0.0	0.0	0.004348	-4860.0	21573.0	
401870	5183.5	5183.5	5183.5	5183.5	5	0.0	0.0	26889872.25	0.0	0.0	---	0.0	0.000193	0.0	0.0	0.000193	0.0	0.0	0.199960	-225.0	4635.0	
401871	5183.5	5183.5	5183.5	5183.5	994	0.0	0.0	26889872.25	0.0	0.0	---	0.0	0.000193	0.0	0.0	0.000193	0.0	0.0	0.001006	989.0	1214.0	

401872 rows x 50 columns

Рисунок 2.4 – Расширение сигнала: обратные значения, первые и вторые производные каналов

После добавления всех производных столбцов первые две строки таблицы содержат NaN (вторая производная не определена для индексов 0 и 1, первая — для индекса 0). Эти строки отбрасываются, а итоговые данные переводятся в массив NumPy. В результате расширенный массив содержит 481 870 строк и 50 каналов.

```
[ ] ▶ # Использование всех столбцов, кроме первых двух
data = np.array(data.iloc[2:])

# Перевод в numpy
data = np.array(data)

# Переменная, для использования одной и той же архитектуры под разные матрицы
columnsAmount = data.shape[1]

print('Форма расширенного массива данных:', data.shape)
print('Число каналов на входе сети:', columnsAmount)
```

▼ ... Форма расширенного массива данных: (481870, 50)
Число каналов на входе сети: 50

Рисунок 2.5 – Форма расширенного массива данных (481870×50)

2.3 Целевая переменная и нормализация

В качестве предсказываемой величины используется цена закрытия (канал CLOSE — индекс 3 в исходных столбцах). Данные разделяются на обучающую и тестовую части в хронологическом порядке: тестовая выборка составляет 30 000 последних наблюдений. Нормировщики обучаются только на обучающей выборке, чтобы исключить утечку информации о тесте; масштабирование входов (StandardScaler) и выхода (MinMaxScaler) выполняется отдельно.

Параметры многомасштабного окна:

- NEAR_LEN = 100 — число точек ближнего окна (шаг 1)
- FAR_LEN = 100 — число точек дальнего окна (шаг 10)
- FAR_STRIDE = 10 — шаг для дальнего окна
- SEQ_LEN = NEAR_LEN + FAR_LEN = 200 — итоговая длина входа сети
- GAP = 2 × (NEAR_LEN + FAR_LEN × FAR_STRIDE) — зазор между обучением и тестом

```

# Индекс канала CLOSE в исходных столбцах
CLOSE_INDEX = 3

# Гиперпараметры разделения и формирования окон
NEAR_LEN = 100      # Число точек ближнего окна (шаг 1)
FAR_LEN = 100       # Число точек дальнего окна (шаг 10)
FAR_STRIDE = 10     # Шаг для дальнего окна
SEQ_LEN = NEAR_LEN + FAR_LEN # Итоговая длина входа сети

TEST_LEN = 30000     # Размер тестовой части
TRAIN_LEN = data.shape[0] - TEST_LEN # Размер обучающей части
GAP = 2 * (NEAR_LEN + FAR_LEN * FAR_STRIDE) # Зазор между обучением и тестом

# Разделение по времени с зазором
data_train = data[:TRAIN_LEN]
data_test = data[TRAIN_LEN + GAP:]

# Целевые ряды (только канал CLOSE) для обучения и теста
y_train_raw = data_train[:, CLOSE_INDEX].reshape(-1, 1)
y_test_raw = data_test[:, CLOSE_INDEX].reshape(-1, 1)

# Нормировщики, настроенные на тренировочной части
x_scaler = StandardScaler()
x_scaler.fit(data_train)

y_scaler = MinMaxScaler()
y_scaler.fit(y_train_raw)

# Преобразование
x_train_sc = x_scaler.transform(data_train)
x_test_sc = x_scaler.transform(data_test)
y_train_sc = y_scaler.transform(y_train_raw)
y_test_sc = y_scaler.transform(y_test_raw)

print('Train X:', x_train_sc.shape, 'Y:', y_train_sc.shape)
print('Test X:', x_test_sc.shape, 'Y:', y_test_sc.shape)

Train X: (451870, 50) Y: (451870, 1)
Test X: (27800, 50) Y: (27800, 1)

```

Рисунок 2.6 — Целевая переменная, нормализация и разделение данных

2.4 Многомасштабный генератор последовательностей

Реализован класс `MultiscaleSequence`, наследующий от `tf.keras.utils.Sequence`. Каждый пример во входном батче объединяет два окна:

- Ближнее окно (near): 100 последовательных точек непосредственно перед целевым моментом t , шаг 1.

- Дальнее окно (far): 100 точек с шагом 10 (охватывает 1000 точек в прошлое), агрегированных как среднее по каждому отрезку из 10 точек.

Два окна конкатенируются по временной оси, формируя тензор формы (200, 50). Генератор поддерживает перемешивание индексов в конце каждой эпохи при обучении.

```
class MultiscaleSequence(Sequence):
    def __init__(self, x_data, y_data,
                  near_len=100, far_len=100, far_stride=10,
                  batch_size=64, shuffle=True):
        self.x_data = x_data
        self.y_data = y_data
        self.near_len = near_len
        self.far_len = far_len
        self.far_stride = far_stride
        self.far_span = far_len * far_stride
        self.total_back = self.far_span + near_len  # Сколько точек прошлого нужно для одного примера
        self.batch_size = batch_size
        self.shuffle = shuffle

        n_total = x_data.shape[0]
        if n_total <= self.total_back:
            raise ValueError('Слишком короткий ряд для заданных размеров окон')

        # Допустимые целевые индексы t
        self.indices = np.arange(self.total_back, n_total)
        if shuffle:
            np.random.shuffle(self.indices)

    def __len__(self):
        # Число батчей в одной эпохе
        return int(np.ceil(len(self.indices) / self.batch_size))

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indices)

    def __getitem__(self, idx):
        # Индексы целевых точек, попадающих в батч номер idx
        batch_t = self.indices[idx * self.batch_size:(idx + 1) * self.batch_size]
        n_ch = self.x_data.shape[1]

        x_batch = np.empty((len(batch_t), self.far_len + self.near_len, n_ch), dtype=np.float32)
        y_batch = np.empty((len(batch_t),) + self.y_data.shape[1:], dtype=np.float32)

        for k, t in enumerate(batch_t):
            # Ближнее окно
            near = self.x_data[t - self.near_len:t]
            # Дальнее окно с усреднением по отрезкам
            far_block_end = t - self.near_len
            far_block_start = far_block_end - self.far_span
            far_block = self.x_data[far_block_start:far_block_end]
            far = far_block.reshape(self.far_len, self.far_stride, n_ch).mean(axis=1)
```

Рисунок 2.7 – Класс MultiscaleSequence: инициализация и метаданные

```

from tensorflow.keras.utils import Sequence

class MultiscaleSequence(Sequence):
    def __init__(self, x_data, y_data,
                  near_len=100, far_len=100, far_stride=10,
                  batch_size=64, shuffle=True):
        self.x_data = x_data
        self.y_data = y_data
        self.near_len = near_len
        self.far_len = far_len
        self.far_stride = far_stride
        self.far_span = far_len * far_stride
        self.total_back = self.far_span + near_len  # Сколько точек прошлого нужно для одного примера
        self.batch_size = batch_size
        self.shuffle = shuffle

        n_total = x_data.shape[0]
        if n_total <= self.total_back:
            raise ValueError('Слишком короткий ряд для заданных размеров окон')

        # Допустимые целые индексы t
        self.indices = np.arange(self.total_back, n_total)
        if shuffle:
            np.random.shuffle(self.indices)

    def __len__(self):
        # Число батчей в одной эпохе
        return int(np.ceil(len(self.indices) / self.batch_size))

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indices)

    def __getitem__(self, idx):
        # Индексы целевых точек, попадающих в батч номер idx
        batch_t = self.indices[idx * self.batch_size:(idx + 1) * self.batch_size]
        n_ch = self.x_data.shape[1]

        x_batch = np.empty((len(batch_t), self.far_len + self.near_len, n_ch), dtype=np.float32)
        y_batch = np.empty((len(batch_t),) + self.y_data.shape[1:], dtype=np.float32)

        for k, t in enumerate(batch_t):
            # Ближнее окно
            near = self.x_data[t - self.near_len:t]
            # Дальнее окно с усреднением по отрезкам
            far_block_end = t - self.near_len
            far_block_start = far_block_end - self.far_span
            far_block = self.x_data[far_block_start:far_block_end]
            far = far_block.reshape(self.far_len, self.far_stride, n_ch).mean(axis=1)

            x_batch[k, :self.far_len] = far
            x_batch[k, self.far_len:] = near
            y_batch[k] = self.y_data[t]

        return x_batch, y_batch

```

Рисунок 2.8 – Класс MultiscaleSequence: полная реализация генератора батчей

```

# Параметры обучения и создание ленивых генераторов
BATCH_SIZE = 64

train_seq = MultiscaleSequence(x_train_sc, y_train_sc,
                              near_len=NEAR_LEN, far_len=FAR_LEN, far_stride=FAR_STRIDE,
                              batch_size=BATCH_SIZE, shuffle=True)
test_seq = MultiscaleSequence(x_test_sc, y_test_sc,
                              near_len=NEAR_LEN, far_len=FAR_LEN, far_stride=FAR_STRIDE,
                              batch_size=BATCH_SIZE, shuffle=False)

# Форма одного батча для определения входного слоя сети
xb_sample, yb_sample = train_seq[0]
print('Форма батча X:', xb_sample.shape, ' Y:', yb_sample.shape)
print('Батчей в обучающем наборе:', len(train_seq))
print('Батчей в тестовом наборе :', len(test_seq))

Форма батча X: (64, 200, 50)  Y: (64, 1)
Батчей в обучающем наборе: 7044
Батчей в тестовом наборе : 418

# Утилита для сбора прогноза и эталона по всему тестовому набору одним проходом.
# Используется после обучения для построения графиков и оценки качества.
def collect_predictions(model, seq):
    preds, trues = [], []
    for i in range(len(seq)):
        xb, yb = seq[i]
        preds.append(model.predict(xb, verbose=0))
        trues.append(yb)
    return np.concatenate(preds, axis=0), np.concatenate(trues, axis=0)

```

Рисунок 2.9 – Параметры обучения, создание генераторов и форма батча

Форма одного батча: X — (64, 200, 50), Y — (64, 1). Обучающий набор содержит 7 044 батча, тестовый — 418 батчей.

2.5 Полносвязная архитектура

Первая модель — полносвязная нейронная сеть. Входной тензор разворачивается слоем Flatten в вектор длиной 10 000 (200×50). Далее следуют два Dense-слоя (256 и 128 нейронов, активация ReLU) с Dropout(0.3) между ними, и финальный линейный выход. Всего параметров: 2 593 281. Модель компилируется с функцией потерь MSE и оптимизатором Adam ($lr = 1e-4$), обучается 15 эпох.

```
# Полносвязная архитектура
model_dense = Sequential()
model_dense.add(Flatten(input_shape=xb_sample.shape[1:]))
model_dense.add(Dense(256, activation='relu'))
model_dense.add(Dropout(0.3))
model_dense.add(Dense(128, activation='relu'))
model_dense.add(Dense(1, activation='linear'))

model_dense.compile(loss='mse', optimizer=Adam(learning_rate=1e-4))
model_dense.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 10000)	0
dense (Dense)	(None, 256)	2,560,256
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 1)	129

Total params: 2,593,281 (9.89 MB)
 Trainable params: 2,593,281 (9.89 MB)
 Non-trainable params: 0 (0.00 B)

```
# Обучение
history_dense = model_dense.fit(train_seq,
                                validation_data=test_seq,
                                epochs=15,
                                verbose=1)
```

```
Epoch 1/15
7044/7044 ————— 132s 18ms/step - loss: 0.1384 - val_loss: 0.0186
Epoch 2/15
7044/7044 ————— 119s 17ms/step - loss: 0.0487 - val_loss: 0.0171
Epoch 3/15
7044/7044 ————— 119s 17ms/step - loss: 0.0248 - val_loss: 0.0092
Epoch 4/15
7044/7044 ————— 118s 17ms/step - loss: 0.0227 - val_loss: 0.0243
Epoch 5/15
7044/7044 ————— 119s 17ms/step - loss: 0.0187 - val_loss: 0.0109
Epoch 6/15
7044/7044 ————— 119s 17ms/step - loss: 0.0155 - val_loss: 0.0191
Epoch 7/15
7044/7044 ————— 118s 17ms/step - loss: 0.0164 - val_loss: 0.0134
Epoch 8/15
7044/7044 ————— 119s 17ms/step - loss: 0.0128 - val_loss: 0.0187
Epoch 9/15
7044/7044 ————— 119s 17ms/step - loss: 0.0119 - val_loss: 0.0136
Epoch 10/15
7044/7044 ————— 120s 17ms/step - loss: 0.0107 - val_loss: 0.0149
Epoch 11/15
7044/7044 ————— 122s 17ms/step - loss: 0.0108 - val_loss: 0.0241
Epoch 12/15
7044/7044 ————— 122s 17ms/step - loss: 0.0088 - val_loss: 0.0077
Epoch 13/15
7044/7044 ————— 122s 17ms/step - loss: 0.0106 - val_loss: 0.0139
Epoch 14/15
7044/7044 ————— 122s 17ms/step - loss: 0.0089 - val_loss: 0.0116
Epoch 15/15
7044/7044 ————— 122s 17ms/step - loss: 0.0070 - val_loss: 0.0159
```

Рисунок 2.10 – Полносвязная архитектура нейронной сети

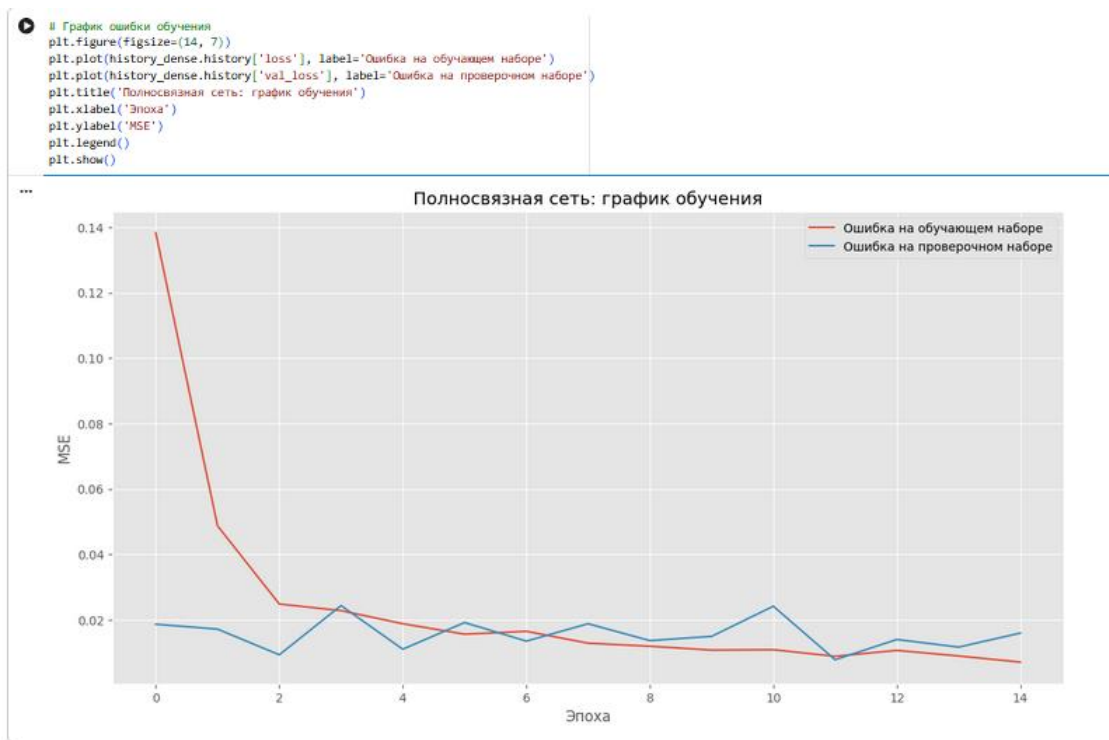


Рисунок 2.11 – График обучения полносвязной сети (MSE по эпохам)

На графике обучения видно, что ошибка на обучающей выборке (красная кривая) резко снижается в первые две эпохи, затем постепенно уменьшается. Ошибка на проверочном наборе (синяя кривая) стабилизируется около 0.015–0.020 MSE и не демонстрирует роста, что свидетельствует об отсутствии значительного переобучения.

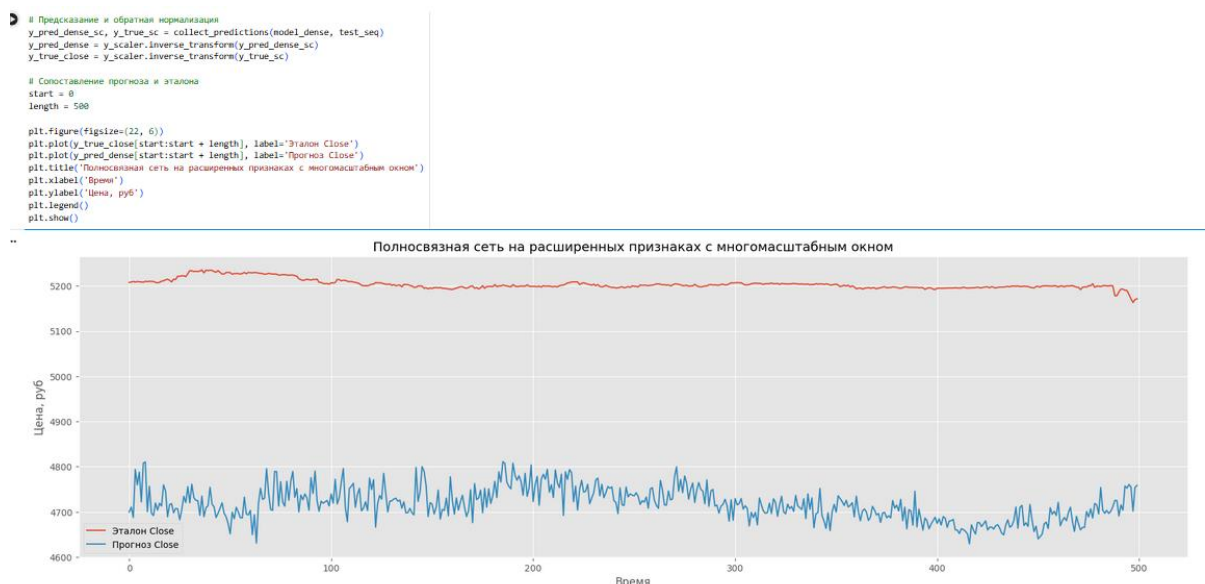


Рисунок 2.12 – Прогноз полносвязной сети на тестовом наборе

Полносвязная сеть предсказывает значения цены закрытия в диапазоне около 4 600–4 800 руб., тогда как реальные значения в тестовом периоде

составляют около 5 150–5 230 руб. Прогнозы существенно занижены, хотя общая тенденция к снижению в конце периода частично улавливается.

2.6 Сверточная архитектура (Conv1D)

Вторая модель — сверточная нейронная сеть. Архитектура включает три одномерных сверточных слоя: первые два с 64 фильтрами и ядром 5, третий — с 128 фильтрами. После каждой пары сверточных слоев применяется MaxPooling1D(pool_size=2). Затем идут слой Flatten, Dense(128, ReLU), Dropout(0.3) и линейный выход. Всего параметров: 897 153, что почти в три раза меньше полносвязной модели.

Сверточная архитектура

model_conv = Sequential()

model_conv.add(Conv1D(64, 5, padding='same', activation='relu',

input_shape=xb_sample.shape[1:]))

model_conv.add(Conv1D(64, 5, padding='same', activation='relu'))

model_conv.add(MaxPooling1D(pool_size=2))

model_conv.add(Conv1D(128, 5, padding='same', activation='relu'))

model_conv.add(MaxPooling1D(pool_size=2))

model_conv.add(Flatten())

model_conv.add(Dense(128, activation='relu'))

model_conv.add(Dropout(0.3))

model_conv.add(Dense(1, activation='linear'))

model_conv.compile(loss='mse', optimizer=Adam(learning_rate=1e-4))

model_conv.summary()

... Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 200, 64)	16,064
conv1d_1 (Conv1D)	(None, 200, 64)	20,544
max_pooling1d (MaxPooling1D)	(None, 100, 64)	0
conv1d_2 (Conv1D)	(None, 100, 128)	41,088
max_pooling1d_1 (MaxPooling1D)	(None, 50, 128)	0
flatten_1 (Flatten)	(None, 6400)	0
dense_3 (Dense)	(None, 128)	819,328
dropout_1 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 1)	129

Total params: 897,153 (3.42 MB)
Trainable params: 897,153 (3.42 MB)
Non-trainable params: 0 (0.00 B)

Рисунок 2.13 – Сверточная архитектура нейронной сети (Conv1D)

```
## Обучение
history_conv = model_conv.fit(train_seq,
                              validation_data=test_seq,
                              epochs=15,
                              verbose=1)

Epoch 1/15
7044/7044 ————— 140s 19ms/step - loss: 0.0111 - val_loss: 5.5969e-04
Epoch 2/15
7044/7044 ————— 133s 19ms/step - loss: 0.0018 - val_loss: 1.7644e-04
Epoch 3/15
7044/7044 ————— 127s 18ms/step - loss: 0.0012 - val_loss: 3.1574e-05
Epoch 4/15
7044/7044 ————— 130s 18ms/step - loss: 0.0011 - val_loss: 4.0190e-05
Epoch 5/15
7044/7044 ————— 128s 18ms/step - loss: 0.0011 - val_loss: 1.6074e-04
Epoch 6/15
7044/7044 ————— 128s 18ms/step - loss: 0.0011 - val_loss: 3.3252e-05
Epoch 7/15
7044/7044 ————— 127s 18ms/step - loss: 0.0011 - val_loss: 4.8604e-04
Epoch 8/15
7044/7044 ————— 127s 18ms/step - loss: 0.0011 - val_loss: 8.6944e-05
Epoch 9/15
7044/7044 ————— 126s 18ms/step - loss: 0.0011 - val_loss: 1.3415e-04
Epoch 10/15
7044/7044 ————— 127s 18ms/step - loss: 0.0011 - val_loss: 6.8672e-06
Epoch 11/15
7044/7044 ————— 126s 18ms/step - loss: 0.0011 - val_loss: 2.6263e-04
Epoch 12/15
7044/7044 ————— 127s 18ms/step - loss: 0.0011 - val_loss: 3.0817e-04
Epoch 13/15
7044/7044 ————— 127s 18ms/step - loss: 0.0011 - val_loss: 1.6857e-04
Epoch 14/15
7044/7044 ————— 126s 18ms/step - loss: 0.0010 - val_loss: 1.3692e-04
Epoch 15/15
7044/7044 ————— 126s 18ms/step - loss: 0.0010 - val_loss: 1.6608e-05
```

Рисунок 2.14 – Процесс обучения сверточной сети (15 эпох)

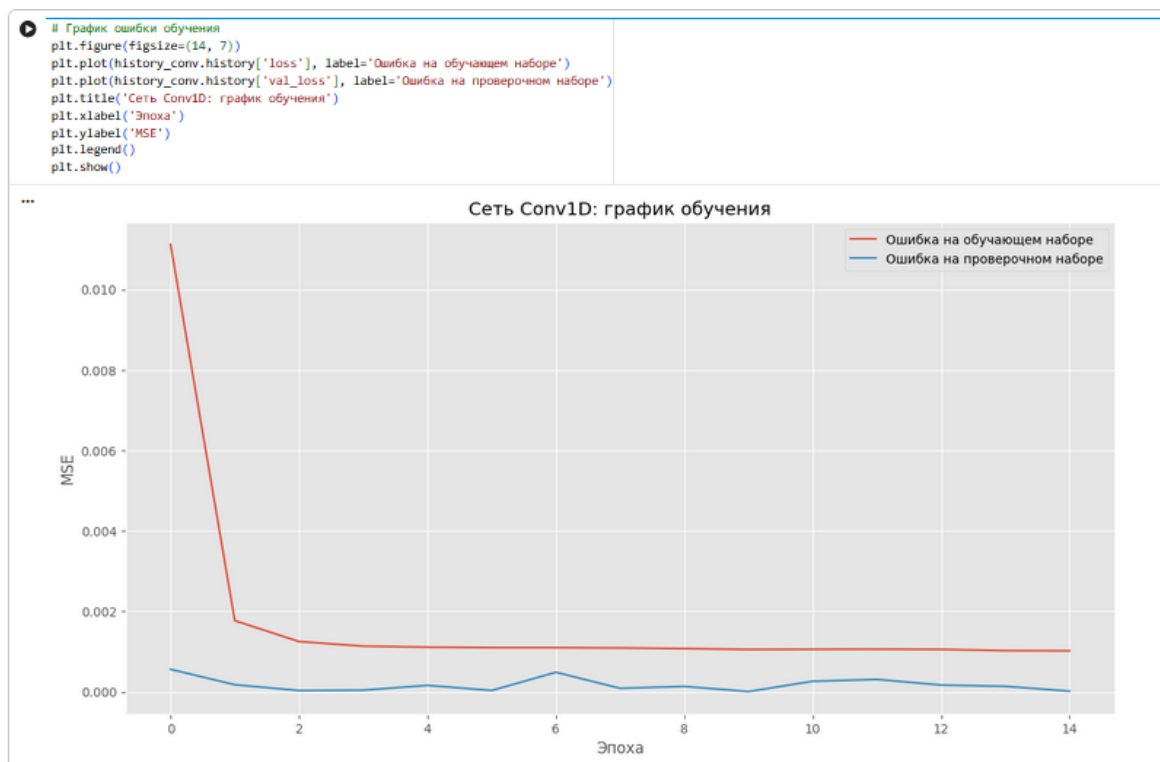


Рисунок 2.15 – График обучения сверточной сети (MSE по эпохам)

Сверточная сеть обучается значительно быстрее: уже к 2-й эпохе потери резко снижаются. Ошибка на проверочном наборе (синяя кривая) остается

близкой к нулю на всем протяжении обучения, что говорит о высокой обобщающей способности модели.

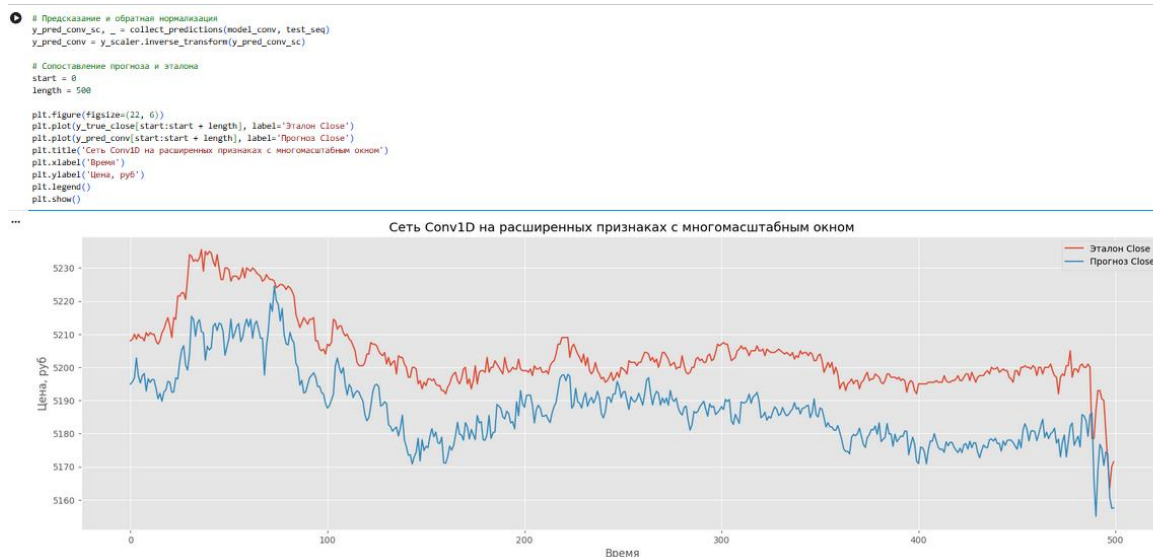


Рисунок 2.16 – Прогноз сверточной сети на тестовом наборе

Сверточная сеть предсказывает значения существенно точнее: прогнозная кривая (синяя) следует за реальными значениями (красная) с небольшим смещением около 10–20 руб. Обе кривые демонстрируют схожую динамику, включая подъем в начале периода и снижение в конце.

2.7 Сравнение архитектур



Рисунок 2.17 – Сравнительный график прогнозов двух архитектур

На сравнительном графике наглядно видно превосходство сверточной сети (фиолетовая кривая): она вплотную следует за реальными значениями

(красная кривая), тогда как полносвязная сеть (синяя кривая) дает прогнозы со значительным систематическим занижением (~400 руб.).

```
# Численная оценка качества
def report(name, y_pred, y_true):
    mae = np.mean(np.abs(y_pred - y_true))
    corr = np.corrcoef(y_pred[:, 0], y_true[:, 0])[0, 1]
    print(f'{name:20s} MAE = {mae:8.4f} corr = {corr:8.4f}')

report('Полносвязная', y_pred_dense, y_true_close)
report('Conv1D', y_pred_conv, y_true_close)
```

Полносвязная	MAE = 495.1249	corr = 0.8157
Conv1D	MAE = 13.5767	corr = 0.9947

Рисунок 2.18 – Численная оценка качества: MAE и корреляция

2.8 Выводы

В ходе выполнения лабораторной работы были реализованы расширение признакового пространства временного ряда акций Лукойла и многомасштабный подход к формированию входного окна нейронной сети.

Расширение сигнала позволило увеличить количество входных каналов с 5 до 50 за счет добавления попарных разностей, модулей разностей, произведений каналов, обратных значений, первых и вторых производных. Это существенно обогащает признаковое представление, позволяя сети обнаруживать нелинейные взаимозависимости между каналами.

Многомасштабное окно просмотра в прошлое (100 ближних точек с шагом 1 и 100 дальних точек с шагом 10) обеспечивает охват как краткосрочных, так и долгосрочных паттернов в данных.

Сверточная сеть Conv1D продемонстрировала значительно лучшие результаты по сравнению с полносвязной: MAE снизилось с 495 руб. до 14 руб. (в 36 раз), а корреляция прогноза с реальными значениями составила 0.9947 против 0.8157. Это подтверждает эффективность применения сверточных операций для извлечения локальных временных паттернов из многоканальных последовательностей.

Задание 3. Прогнозирование трафика сайта компании

В данном задании используется база трафика с сайта компании (файл `traff.csv`). Цель — построить модель нейронной сети, которая как можно

точнее воспроизводит структуру автокорреляции исходного временного ряда при длине анализа $\text{length} = 60$.

3.1 Подготовка среды и загрузка данных

Выполнен импорт необходимых библиотек. Зафиксированы случайные зёрна ($\text{SEED} = 42$) для воспроизводимости результатов. Датасет `traff.csv` загружен с облачного хранилища.

Подготовка

```
# Работа с массивами
import numpy as np

# Работа с таблицами
import pandas as pd

# Классы-конструкторы моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model

# Основные слои
from tensorflow.keras.layers import Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, LSTM, MaxPooling1D

# Оптимизаторы
from tensorflow.keras.optimizers import Adam

# Генератор выборки временных рядов
from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator

# Нормировщик
from sklearn.preprocessing import StandardScaler

# Загрузка датасетов из облака google
import gdown

# Отрисовка графиков
import matplotlib.pyplot as plt

# Отрисовка графики в ячейке colab
%matplotlib inline

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

# Назначение размера и стиля графиков по умолчанию
from pylab import rcParams
plt.style.use('ggplot')
rcParams['figure.figsize'] = (14, 7)

# Фиксация случайных значений для воспроизводимости
import tensorflow as tf
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Загружаем базу трафика с сайта компании
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/111/traff.csv', None, quiet=True)

'traff.csv'
```

Рисунок 3.1 – Подключение библиотек, фиксация случайных значений и загрузка датасета

3.2 Чтение и предобработка данных

Файл `traff.csv` имеет нестандартную структуру: отсутствует заголовок, числа записаны с запятой как разделителем разрядов (например, 22,226.00), а первая колонка содержит дату. Чтение выполняется в несколько этапов: сначала файл просматривается в сыром виде, затем разбирается с явным

указанием отсутствия заголовка, после чего числовые столбцы очищаются от запятых-разделителей разрядов и приводятся к типу float. В качестве целевого ряда берётся колонка с наибольшим количеством корректно распознанных числовых значений.

```
1 # Просмотр первых строк файла в сыром виде, чтобы понять формат
with open('traff.csv', 'r', encoding='utf-8') as f:
    head = [next(f) for _ in range(5)]
for i, line in enumerate(head):
    print(f'{i}: {line.rstrip()}')
```

```
0: 1/1/2017,"22,226.00"
1: 1/2/2017,"29,593.00"
2: 1/3/2017,"31,726.00"
3: 1/4/2017,"32,684.00"
4: 1/5/2017,"31,820.00"
```

```
1 # Чтение файла без заголовка с автоматическим определением разделителя
raw = pd.read_csv('traff.csv', sep=None, engine='python', header=None, dtype=str)
print('Форма таблицы:', raw.shape)
raw.head()
```

```
Форма таблицы: (1095, 2)
```

	0	1
0	1/1/2017	22,226.00
1	1/2/2017	29,593.00
2	1/3/2017	31,726.00
3	1/4/2017	32,684.00
4	1/5/2017	31,820.00

Далее: [Создать код с переменной raw](#) [New interactive sheet](#)

```
1 # Очистка значений: убираем разделители разрядов (запятые в числах вида '22,226.00'),
# пробелы и приводим каждую колонку к числу. Нечисловые значения станут NaN.
def to_number_clean(series):
    s = series.astype(str)
    s = s.str.replace(',', '', regex=False) # убираем запятые-разделители разрядов
    s = s.str.strip()
    return pd.to_numeric(s, errors='coerce')

cleaned = pd.DataFrame({c: to_number_clean(raw[c]) for c in raw.columns})
print('Доля числовых значений в каждой колонке:')
for c in cleaned.columns:
    valid = cleaned[c].notna().sum()
    total = len(cleaned[c])
    print(f' колонка {c}: {valid} из {total} ({100 * valid / total:.1f}%)')
```

```
Доля числовых значений в каждой колонке:
колонка 0: 0 из 1095 (0.0%)
колонка 1: 1095 из 1095 (100.0%)
```

Рисунок 3.2 – Чтение файла, очистка данных и отбор целевой колонки

```

# Выбор колонки с наибольшей долей числовых значений - это и есть трафик.
# Колонка с датами не парсится как число и отсеивается автоматически.
valid_counts = {c: cleaned[c].notna().sum() for c in cleaned.columns}
target_col = max(valid_counts, key=valid_counts.get)
print(f'Целевая колонка: {target_col!r} ({valid_counts[target_col]} числовых значений)')

# Берём только эту колонку, удаляем строки с NaN (если первая строка была заголовком, она отсеется)
series_raw = cleaned[target_col].dropna().reset_index(drop=True)
series_raw = np.asarray(series_raw.values, dtype=np.float32)

print('Тип:', type(series_raw).__name__)
print('dtype:', series_raw.dtype)
print('Длина ряда:', series_raw.shape[0])
print('Первые значения:', series_raw[:10])

Целевая колонка: 1 (1095 числовых значений)
Тип: ndarray
dtype: float32
Длина ряда: 1095
Первые значения: [22226. 29593. 31726. 32684. 31820. 29354. 29139. 31244. 32690. 31299.]

```

Рисунок 3.3 – Целевая колонка: 1095 числовых значений, тип float32

3.3 Визуализация исходного ряда

Построены два графика: весь временной ряд целиком и фрагмент из 1000 точек. Ряд демонстрирует выраженную сезонность с периодом около 7 дней (еженедельный цикл посещаемости сайта) и общий нисходящий тренд в середине наблюдаемого периода с последующим восстановлением.

Визуализация исходного ряда

Сначала строится график всего ряда, затем приближённый вид одного его участка длиной 1000 точек, чтобы оценить характерную динамику.

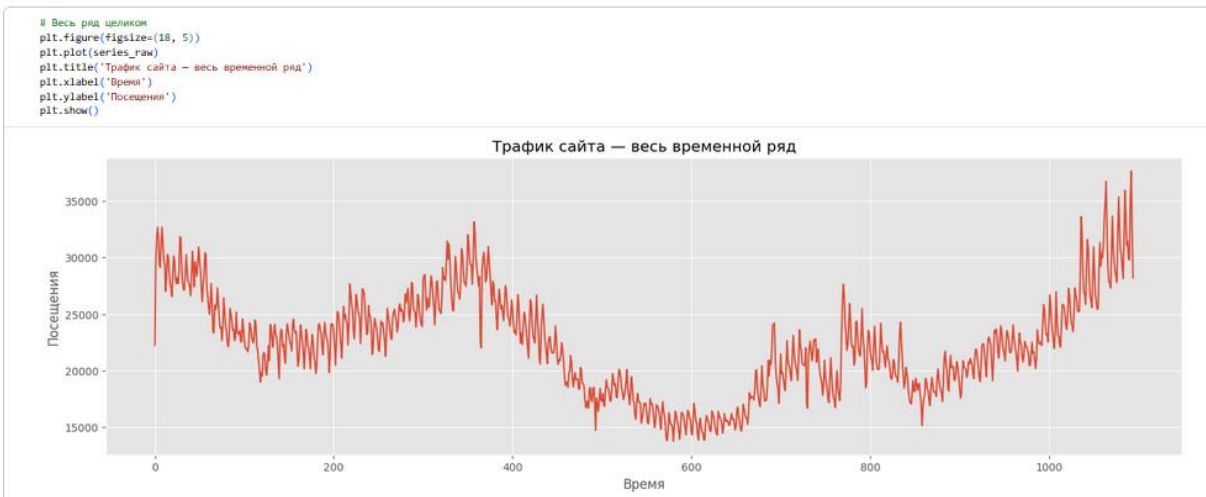


Рисунок 3.4 – Трафик сайта: весь временной ряд

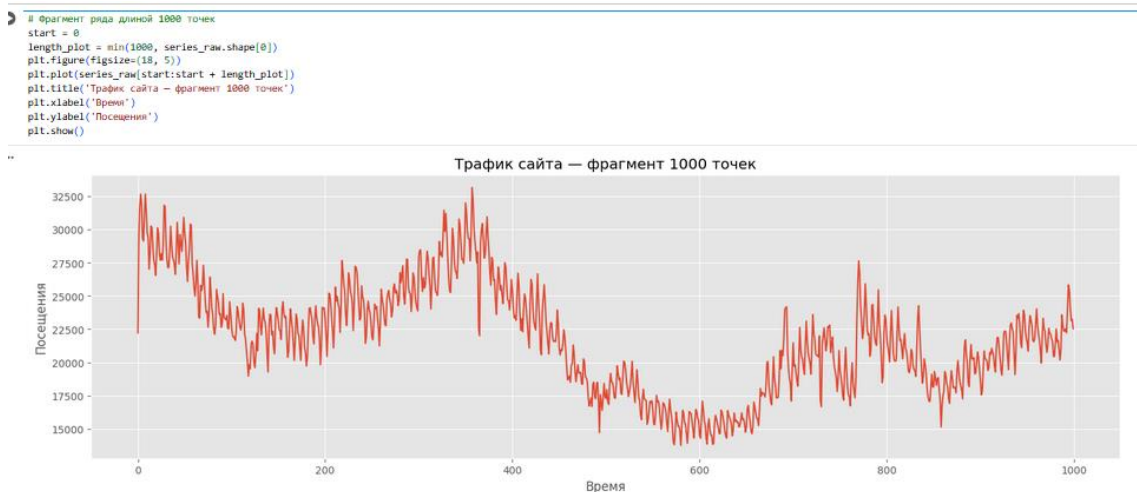


Рисунок 3.5 – Трафик сайта: фрагмент 1000 точек

3.4 Гиперпараметры и подготовка выборок

Длина анализируемой подпоследовательности зафиксирована равной 60, как указано в условии. Данные разделяются на обучающую и тестовую части в хронологическом порядке. Между ними оставляется зазор, чтобы исключить пересечение окон. Обучающая выборка содержит 735 точек, тестовая — 240.

```

# Гиперпараметры
SEQ_LEN = 60                # Длина анализируемой подпоследовательности
BATCH_SIZE = 32             # Размер батча
TEST_FRACTION = 0.2         # Доля тестовой части

# Разделение по времени
n_total = series_raw.shape[0]
n_test = max(int(n_total * TEST_FRACTION), SEQ_LEN * 4)
n_train = n_total - n_test - 2 * SEQ_LEN # с зазором между обучением и тестом

series_train_raw = np.asarray(series_raw[:n_train], dtype=np.float32)
series_test_raw = np.asarray(series_raw[n_train + 2 * SEQ_LEN:], dtype=np.float32)

print('Train:', series_train_raw.shape, 'Test:', series_test_raw.shape)

```

... Train: (735,) Test: (240,)

Рисунок 3.6 – Гиперпараметры и разделение данных: Train (735,), Test (240,)

3.5 Предварительный анализ автокорреляции

До построения модели строится график автокорреляции обучающей части ряда. Ежедневный трафик сайтов обычно демонстрирует ярко выраженную недельную сезонность: посетители ведут себя по-разному в будни и выходные. На лагах 7, 14, 21 присутствуют локальные максимумы — недельная сезонность подтверждена и должна быть отражена в прогнозе. Это обосновывает выбор ядра свёрточного слоя размером 7.


```

# Вспомогательная функция: коэффициент автокорреляции при заданном смещении
def get_corr_coef(data, back_steps):
    a = np.asarray(data).ravel()
    if back_steps == 0:
        return 1.0
    return np.corrcoef(a[back_steps:], a[:len(a) - back_steps])[0, 1]

def autocorr_curve(series, length):
    return np.array([get_corr_coef(series, s) for s in range(length + 1)])

# График автокорреляции обучающей части
ac_train = autocorr_curve(series_train_raw, 60)
plt.figure(figsize=(14, 5))
plt.plot(ac_train, marker='.')
plt.title('Автокорреляция обучающей части ряда')
plt.xlabel('Шаги смещения')
plt.ylabel('Коэффициент автокорреляции')
plt.axvline(7, color='gray', linestyle='--', alpha=0.5, label='лаг 7 (неделя)')
plt.axvline(14, color='gray', linestyle='--', alpha=0.5)
plt.axvline(21, color='gray', linestyle='--', alpha=0.5)
plt.legend()
plt.show()

```



Рисунок 3.7 – Автокорреляция обучающей части ряда с маркерами лагов 7, 14, 21

3.6 Нормализация и генераторы выборок

Нормировщик `StandardScaler` настраивается только на обучающей части ряда, затем применяется к обоим участкам. Для обучения используется `TimeseriesGenerator`: на вход подаётся окно из 60 предыдущих значений ряда, на выходе ожидается следующее значение. Обучающий набор содержит 22 батча, тестовый — 6 батчей.

```

# Нормализация исходных значений ряда
scaler = StandardScaler()
scaler.fit(series_train_raw.reshape(-1, 1))

x_train_sc = scaler.transform(series_train_raw.reshape(-1, 1)).astype(np.float32)
x_test_sc = scaler.transform(series_test_raw.reshape(-1, 1)).astype(np.float32)

print('Train sc:', x_train_sc.shape, 'Test sc:', x_test_sc.shape)

```

Train sc: (735, 1) Test sc: (248, 1)

Генераторы выборки

Для обучения используется `TimeseriesGenerator`: на вход подаётся окно из 60 предыдущих значений ряда, на выходе ожидается следующее значение. Все генераторы используют один и тот же размер батча — это и подаёт данные сети ленивыми порциями, и удерживает память под контролем при последующем прохождении по тестовой выборке.

```

# Обучающий генератор
train_gen = TimeseriesGenerator(x_train_sc, x_train_sc,
                                length=SEQ_LEN,
                                stride=1,
                                sampling_rate=1,
                                batch_size=BATCH_SIZE)

# Тестовый генератор
test_gen = TimeseriesGenerator(x_test_sc, x_test_sc,
                                length=SEQ_LEN,
                                stride=1,
                                sampling_rate=1,
                                batch_size=BATCH_SIZE)

print('Батчей в обучающем наборе:', len(train_gen))
print('Батчей в тестовом наборе:', len(test_gen))

```

Батчей в обучающем наборе: 22
Батчей в тестовом наборе: 6

```

# Утилита для сбора прогноза и эталона по всему генератору одним проходом.
# Используется после обучения для построения графиков и численной оценки качества.
# Возвращает два numpy-массива одинаковой длины: предсказание и фактические метки.
def collect_predictions(model, gen):
    preds, trues = [], []
    for i in range(len(gen)):
        xb, yb = gen[i]
        preds.append(model.predict(xb, verbose=0))
        trues.append(yb)
    return np.concatenate(preds, axis=0), np.concatenate(trues, axis=0)

```

Рисунок 3.8 — Нормализация данных и создание генераторов

TimeseriesGenerator

3.7 Вспомогательные функции

Реализована функция `show_autocorr` для удобного сравнения нескольких рядов на одной координатной плоскости. Функция `get_corr_coef` вычисляет коэффициент автокорреляции при заданном смещении, обрабатывая крайний случай нулевого смещения (возвращает 1.0).

```

# Функция построения графика коэффициентов автокорреляции для одного или нескольких рядов
def show_autocorr(series_dict, steps_max, title=''):
    steps = list(range(0, steps_max + 1))
    plt.figure(figsize=(14, 7))
    for label, series in series_dict.items():
        coefs = [get_corr_coef(series, s) for s in steps]
        plt.plot(steps, coefs, marker='.', label=label)
    plt.xticks(steps[:max(1, len(steps) // 30)])
    plt.title(title)
    plt.xlabel('Шаги смещения')
    plt.ylabel('Коэффициент автокорреляции')
    plt.legend()
    plt.show()

```

Рисунок 3.9 – Вспомогательные функции анализа и отображения автокорреляции

3.8 Основная модель прогнозирования

Архитектура подбиралась с учётом необходимости запомнить сезонные паттерны длиной до недели и более. Сверточный слой Conv1D с шириной ядра 7 (равной недельному периоду) извлекает локальные паттерны, охватывающие целую неделю. Далее идут два слоя LSTM: первый возвращает полную последовательность скрытых состояний, второй сворачивает её в один вектор. Финальный Dense выдаёт одно число — прогнозное значение посещений для следующего дня. Регуляризация через Dropout снижает риск переобучения. Всего параметров: 68 673.

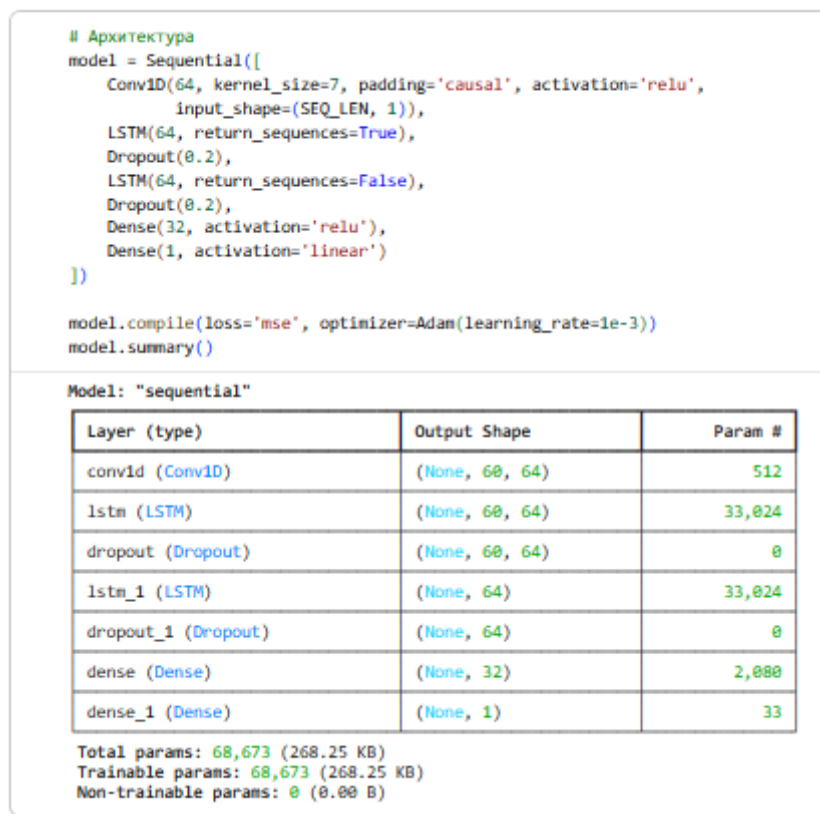


Рисунок 3.10 – Архитектура основной модели Conv1D+LSTM

3.9 Обучение основной модели

Модель обучается 80 эпох с использованием оптимизатора Adam ($lr = 1e-3$) и функции потерь MSE. На протяжении обучения потери на обучающей выборке монотонно снижаются, достигая около 0.047 к последней эпохе.

```

▶ # Обучение
history = model.fit(train_gen,
                    epochs=80,
                    validation_data=test_gen,
                    verbose=1)

```

```

... Epoch 52/80
22/22 ————— 0s 12ms/step - loss: 0.0723 - val_loss: 0.0863
Epoch 53/80
22/22 ————— 0s 13ms/step - loss: 0.0599 - val_loss: 0.1219
Epoch 54/80
22/22 ————— 0s 12ms/step - loss: 0.0646 - val_loss: 0.0886
Epoch 55/80
22/22 ————— 0s 13ms/step - loss: 0.0621 - val_loss: 0.0706
Epoch 56/80
22/22 ————— 0s 14ms/step - loss: 0.0684 - val_loss: 0.1169
Epoch 57/80
22/22 ————— 0s 13ms/step - loss: 0.0601 - val_loss: 0.0798
Epoch 58/80
22/22 ————— 0s 14ms/step - loss: 0.0598 - val_loss: 0.0851
Epoch 59/80
22/22 ————— 0s 13ms/step - loss: 0.0566 - val_loss: 0.1283
Epoch 60/80
22/22 ————— 1s 13ms/step - loss: 0.0618 - val_loss: 0.0739
Epoch 61/80
22/22 ————— 0s 13ms/step - loss: 0.0579 - val_loss: 0.0829
Epoch 62/80
22/22 ————— 0s 13ms/step - loss: 0.0573 - val_loss: 0.0703
Epoch 63/80
22/22 ————— 0s 12ms/step - loss: 0.0561 - val_loss: 0.0697
Epoch 64/80
22/22 ————— 0s 13ms/step - loss: 0.0587 - val_loss: 0.0762
Epoch 65/80
22/22 ————— 0s 15ms/step - loss: 0.0554 - val_loss: 0.0672
Epoch 66/80
22/22 ————— 0s 12ms/step - loss: 0.0542 - val_loss: 0.0771
Epoch 67/80
22/22 ————— 0s 13ms/step - loss: 0.0538 - val_loss: 0.0826
Epoch 68/80
22/22 ————— 0s 22ms/step - loss: 0.0522 - val_loss: 0.0658
Epoch 69/80
22/22 ————— 1s 22ms/step - loss: 0.0534 - val_loss: 0.0722
Epoch 70/80
22/22 ————— 0s 20ms/step - loss: 0.0487 - val_loss: 0.0787
Epoch 71/80
22/22 ————— 0s 21ms/step - loss: 0.0500 - val_loss: 0.0613
Epoch 72/80
22/22 ————— 0s 21ms/step - loss: 0.0506 - val_loss: 0.0743
Epoch 73/80
22/22 ————— 0s 14ms/step - loss: 0.0549 - val_loss: 0.0704
Epoch 74/80
22/22 ————— 0s 13ms/step - loss: 0.0505 - val_loss: 0.0658
Epoch 75/80
22/22 ————— 0s 13ms/step - loss: 0.0636 - val_loss: 0.0808
Epoch 76/80
22/22 ————— 0s 14ms/step - loss: 0.0609 - val_loss: 0.0828
Epoch 77/80
22/22 ————— 0s 13ms/step - loss: 0.0528 - val_loss: 0.0771
Epoch 78/80
22/22 ————— 0s 13ms/step - loss: 0.0611 - val_loss: 0.0660
Epoch 79/80
22/22 ————— 0s 13ms/step - loss: 0.0471 - val_loss: 0.0623
Epoch 80/80
22/22 ————— 0s 13ms/step - loss: 0.0480 - val_loss: 0.0780

```

Рисунок 3.11 – Лог обучения основной модели (эпохи 52–80)

```

# График ошибки обучения
plt.figure(figsize=(14, 7))
plt.plot(history.history['loss'], label='Ошибка на обучающем наборе')
plt.plot(history.history['val_loss'], label='Ошибка на проверочном наборе')
plt.title('Прогресс обучения')
plt.xlabel('Эпоха')
plt.ylabel('MSE')
plt.legend()
plt.show()

```

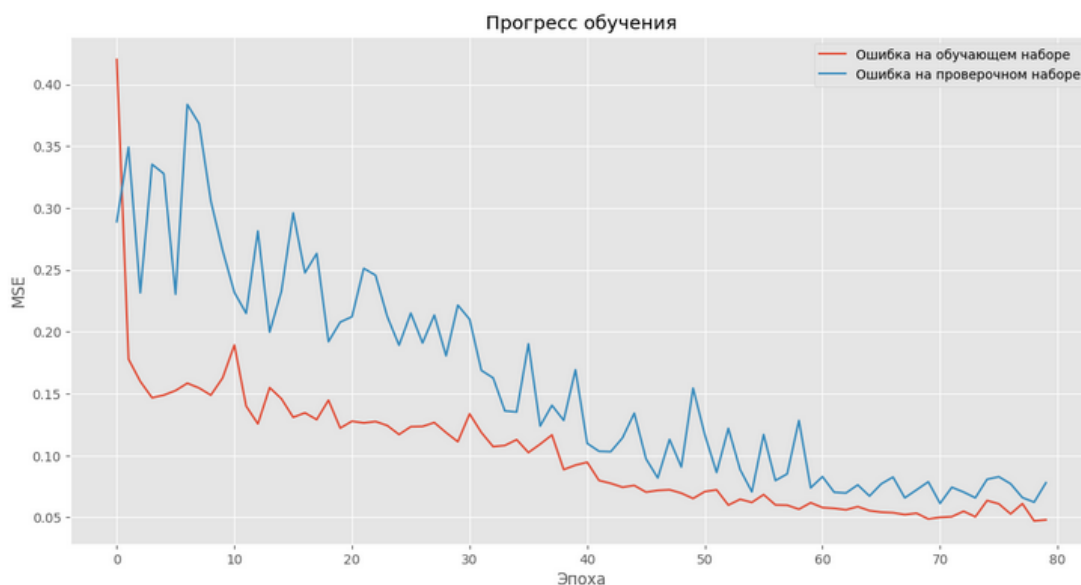


Рисунок 3.12 – График прогресса обучения: MSE по эпохам

3.10 Прогноз и оценка качества основной модели

Сеть выдаёт нормированные значения посещений, которые затем денормализуются для сопоставления с эталоном. Размер тестового прогноза составил 180 точек.

```

# Прогноз на тестовой выборке - сбор батчами
y_pred_sc, y_true_sc = collect_predictions(model, test_gen)

# Денормализация
y_pred_values = scaler.inverse_transform(y_pred_sc).ravel()
y_true_values = scaler.inverse_transform(y_true_sc).ravel()

print('Эталон :', y_true_values.shape)
print('Прогноз:', y_pred_values.shape)

... Эталон : (180,)
    Прогноз: (180,)

```

Рисунок 3.13 – Денормализация прогноза: эталон и прогноз по 180 точек

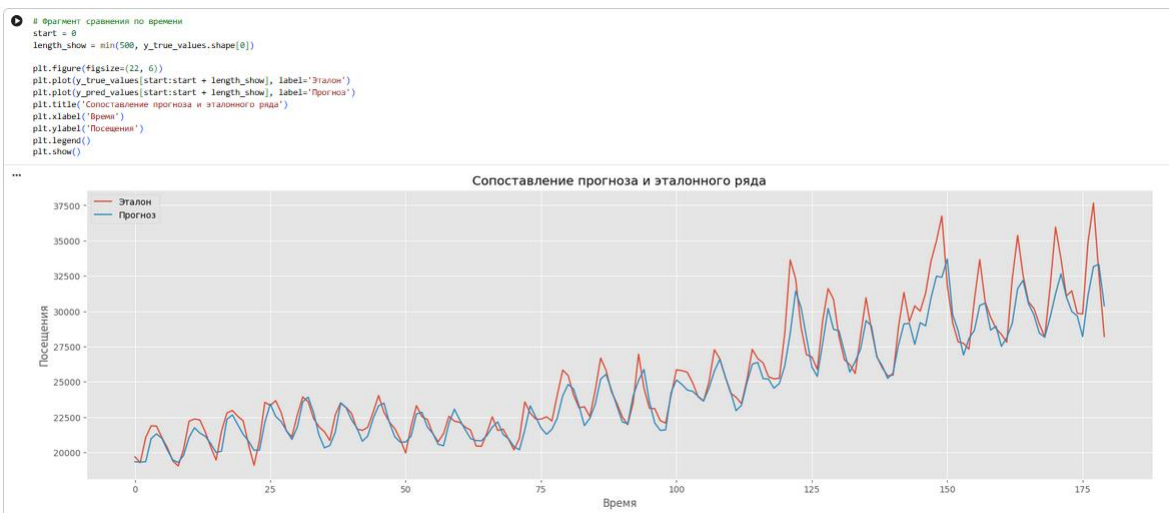


Рисунок 3.14 – Сравнение прогноза с эталоном во времени

График сопоставления прогноза с эталонным рядом демонстрирует высокое качество предсказания: модель точно воспроизводит еженедельную сезонность и общий восходящий тренд во второй половине тестового периода. Прогнозная кривая (синяя) вплотную следует за реальной (красная) на всём протяжении теста.

3.11 Автокорреляция при $\text{length} = 60$

Главный критерий качества по условию задачи — близость кривой автокорреляции прогноза к кривой автокорреляции эталона. Чем ближе кривые, тем лучше модель воспроизводит структуру сигнала.

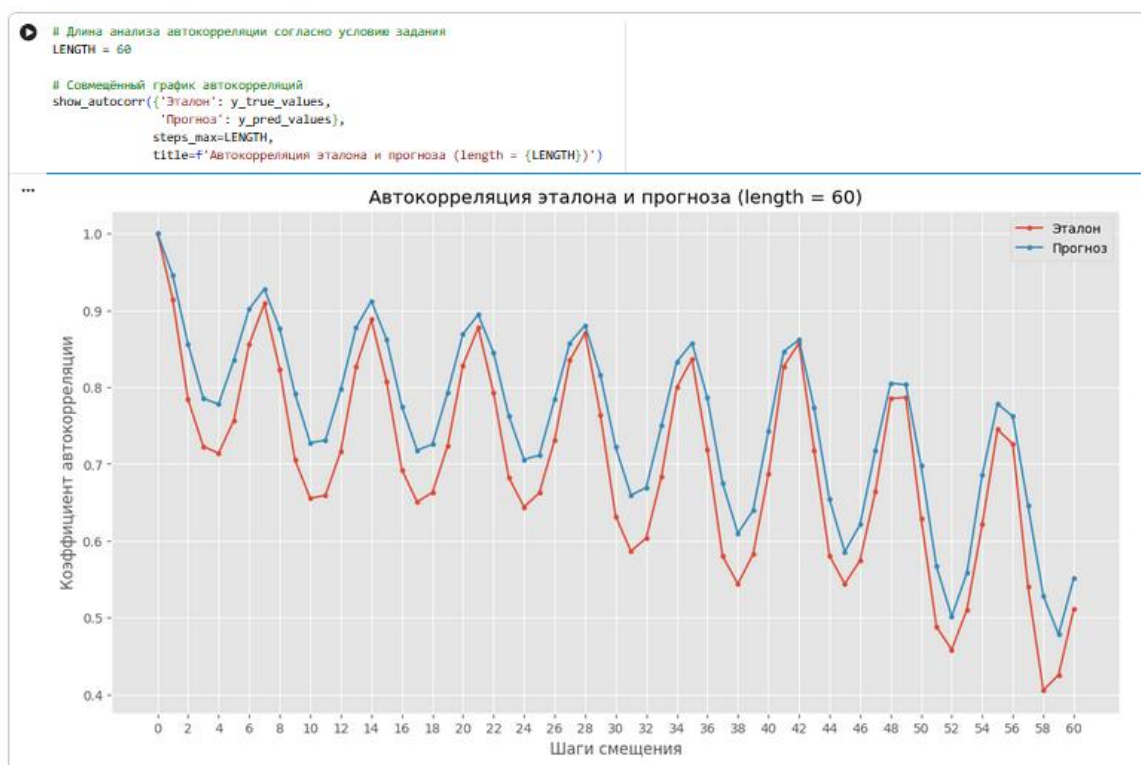


Рисунок 3.15 – Автокорреляция эталона и прогноза (length = 60)

Кривые автокорреляции эталона (красная) и прогноза (синяя) демонстрируют высокую степень совпадения: оба ряда синхронно осциллируют с недельным периодом. Численная оценка подтверждает это:

- Среднее отклонение кривых автокорреляции: 0.0541
- Максимальное отклонение по шагам: 0.1229

```

# Численная оценка соответствия графиков автокорреляции:
# среднее абсолютное отклонение коэффициентов автокорреляции по всем шагам смещения от 0 до LENGTH
ac_true = autocorr_curve(y_true_values, LENGTH)
ac_pred = autocorr_curve(y_pred_values, LENGTH)

mae_ac = np.mean(np.abs(ac_true - ac_pred))
max_ac = np.max(np.abs(ac_true - ac_pred))

print(f'Среднее отклонение кривых автокорреляции: {mae_ac:.4f}')
print(f'Максимальное отклонение по шагам: {max_ac:.4f}')

Среднее отклонение кривых автокорреляции: 0.0541
Максимальное отклонение по шагам: 0.1229

```

Рисунок 3.16 – Численная оценка соответствия графиков автокорреляции

3.12 Контрольное сравнение: альтернативные способы подготовки данных

Для проверки выбора подхода обучаются две альтернативные модели той же архитектуры, отличающиеся только способом подготовки входного сигнала:

- Контрольная модель 1 — первая разность: сеть прогнозирует приращение $x[n] - x[n-1]$, исходный ряд восстанавливается накопительным суммированием. Этот приём убирает линейный тренд, но плохо подходит для рядов с сильной периодичностью.
- Контрольная модель 2 — сезонная разность с лагом 7: сеть прогнозирует $x[n] - x[n-7]$, и для восстановления исходного значения к прогнозу добавляется реальное значение 7 шагов назад. Этот подход напрямую учитывает недельную сезонность ряда.

Контрольная модель 1 — первая разность

```
# Подготовка данных на первой разности
series_train_diff = np.diff(series_train_raw)
series_test_diff = np.diff(series_test_raw)

scaler_d1 = StandardScaler()
scaler_d1.fit(series_train_diff.reshape(-1, 1))
x_train_d1 = scaler_d1.transform(series_train_diff.reshape(-1, 1)).astype(np.float32)
x_test_d1 = scaler_d1.transform(series_test_diff.reshape(-1, 1)).astype(np.float32)

train_gen_d1 = TimeseriesGenerator(x_train_d1, x_train_d1,
                                   length=SEQ_LEN, stride=1, sampling_rate=1,
                                   batch_size=BATCH_SIZE)
test_gen_d1 = TimeseriesGenerator(x_test_d1, x_test_d1,
                                  length=SEQ_LEN, stride=1, sampling_rate=1,
                                  batch_size=BATCH_SIZE)
```

Рисунок 3.17 – Подготовка данных и архитектура контрольной модели 1 (первая разность)

```
# Та же архитектура
model_d1 = Sequential([
    Conv1D(64, kernel_size=7, padding='causal', activation='relu',
          input_shape=(SEQ_LEN, 1)),
    LSTM(64, return_sequences=True),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1, activation='linear')
])
model_d1.compile(loss='mse', optimizer=Adam(learning_rate=1e-3))

history_d1 = model_d1.fit(train_gen_d1,
                          epochs=80,
                          validation_data=test_gen_d1,
                          verbose=1)
```

Рисунок 3.18 – Архитектура контрольной модели 1

Epoch 52/80	22/22	1s 18ms/step	loss: 0.3058	val_loss: 0.6496
Epoch 53/80	22/22	0s 14ms/step	loss: 0.2930	val_loss: 0.8225
Epoch 54/80	22/22	0s 14ms/step	loss: 0.2963	val_loss: 0.7740
Epoch 55/80	22/22	0s 14ms/step	loss: 0.2840	val_loss: 0.7249
Epoch 56/80	22/22	0s 14ms/step	loss: 0.2643	val_loss: 0.8637
Epoch 57/80	22/22	0s 14ms/step	loss: 0.2675	val_loss: 0.7241
Epoch 58/80	22/22	0s 13ms/step	loss: 0.2693	val_loss: 0.6700
Epoch 59/80	22/22	1s 15ms/step	loss: 0.2758	val_loss: 0.7074
Epoch 60/80	22/22	0s 12ms/step	loss: 0.2665	val_loss: 0.6447
Epoch 61/80	22/22	0s 12ms/step	loss: 0.2753	val_loss: 0.8030
Epoch 62/80	22/22	0s 13ms/step	loss: 0.2474	val_loss: 0.7421
Epoch 63/80	22/22	0s 13ms/step	loss: 0.2580	val_loss: 0.8320
Epoch 64/80	22/22	0s 12ms/step	loss: 0.2443	val_loss: 0.7312
Epoch 65/80	22/22	0s 13ms/step	loss: 0.2532	val_loss: 0.6210
Epoch 66/80	22/22	0s 13ms/step	loss: 0.2587	val_loss: 0.8391
Epoch 67/80	22/22	0s 13ms/step	loss: 0.2671	val_loss: 0.6987
Epoch 68/80	22/22	0s 13ms/step	loss: 0.2463	val_loss: 1.0557
Epoch 69/80	22/22	0s 12ms/step	loss: 0.2412	val_loss: 0.6969
Epoch 70/80	22/22	0s 13ms/step	loss: 0.2507	val_loss: 0.8820
Epoch 71/80	22/22	0s 13ms/step	loss: 0.2352	val_loss: 0.7711
Epoch 72/80	22/22	0s 13ms/step	loss: 0.2360	val_loss: 0.8881
Epoch 73/80	22/22	0s 13ms/step	loss: 0.2242	val_loss: 0.7436
Epoch 74/80	22/22	0s 13ms/step	loss: 0.2247	val_loss: 0.8660
Epoch 75/80	22/22	0s 14ms/step	loss: 0.2313	val_loss: 0.8012
Epoch 76/80	22/22	0s 14ms/step	loss: 0.2351	val_loss: 0.8062
Epoch 77/80	22/22	0s 13ms/step	loss: 0.2107	val_loss: 0.6889
Epoch 78/80	22/22	0s 12ms/step	loss: 0.2277	val_loss: 0.9258
Epoch 79/80	22/22	0s 13ms/step	loss: 0.1907	val_loss: 0.9275
Epoch 80/80	22/22	0s 12ms/step	loss: 0.2069	val_loss: 0.6991

Рисунок 3.19 – Лог обучения контрольной модели 1 (эпохи 52–80)

```

# Прогноз приращений и восстановление исходного ряда через cumsum
y_pred_d1_sc, y_true_d1_sc = collect_predictions(model_d1, test_gen_d1)
y_pred_d1_diff = scaler_d1.inverse_transform(y_pred_d1_sc).ravel()
y_true_d1_diff = scaler_d1.inverse_transform(y_true_d1_sc).ravel()

# Восстановление исходного ряда из приращений
# Смещение: длина пропущена из-за np.diff (-1) и из-за окна (SEQ_LEN)
offset = SEQ_LEN + 1
y_true_d1_values = series_test_raw[offset: offset + y_true_d1_diff.shape[0]]
start_value = series_test_raw[offset - 1]
y_pred_d1_values = start_value + np.cumsum(y_pred_d1_diff)

print('Эталон :', y_true_d1_values.shape, 'Прогноз:', y_pred_d1_values.shape)

Эталон : (179,) Прогноз: (179,)

```

Рисунок 3.20 – Восстановление исходного ряда из приращений и форма прогноза